Zero-Knowledge Proofs: Removing Interaction from Sigma Protocols in the Non-programmable Random Oracle Model with an Online Extractor

Ross Bathgate

4th Year Project Report
Computer Science and Mathematics
School of Informatics
University of Edinburgh

2025

Abstract

A zero-knowledge proof is a method of proving a statement without revealing anything other than the fact the statement is true. This seemingly paradoxical construction has become increasingly popular and widely applicable, with applications to electronic voting and auctions, cryptographic key distribution, digital banking, and many others.

This project aims to build on the work of Kondi and Shelat [1], taking inspiration from Ciampi [2], to achieve a non-interactive protocol (proof of knowledge) with preferable security properties and assumptions that better match the real-world than many alternative protocols. A new transform is constructed, and an implementation is provided (in Python) to compare runtime against existing approaches. Under certain assumptions, the construction runs efficiently in polynomial time and empirical results indicate that the new construction outperforms an alternative approach of Fischlin [3] upon which much of the project is based.

Acknowledgements

I would like to thank Dr. Michele Ciampi for his invaluable guidance and support throughout this project, as well as my family and friends for their constant encouragement.

Table of Contents

Chapter 1

Introduction

A Zero-Knowledge Proof (ZKP) is a mechanism for a prover P to convince a verifier V that a statement x is true, but without revealing anything other than the fact the statement is true. At first, this idea seems paradoxical, but the following simple example illustrates the concept.

Example 1 *Suppose the prover claims to the verifier that they have discovered a hidden treasure chest. To prove this without revealing its location, the prover blindfolds the verifier and guides them to the treasure, possibly avoiding the shortest route to deceive the verifier. Once there, the prover removes the blindfold just long enough for the verifier to see the treasure but not gather any information about its surroundings. The verifier is now convinced that the treasure exists, yet they have learned nothing about where it is.*

In ZKPs we assume the prover is not computationally bounded (it could take a very long time to establish whether a statement is true), but the verifier must be efficient and run in polynomial time (it should be quick to establish whether a proof is valid). We are interested in non-trivial languages of statements, i.e. statements for which it is not possible to check truthiness in polynomial time. Thus, for non-trivial languages, the verifier cannot simply determine if a statement is true on its own.

A classical result by O. Goldreich et al. [4] showed that a ZKP exists for every NP[1] language. Hence, for any problem where a solution can be easily verified, there is a method to convince a verifier that a solution exists while keeping it completely secret.

This makes ZKPs widely applicable. For example, they can be used to prove knowledge of a secret cryptographic key, in secret e-voting to preserve anonymity [5], or in multi-party-communication to prove in zero-knowledge that all parties acted honestly [6].

ZKPs can be either interactive or non-interactive. In an interactive ZKP, several rounds of back-and-forth messages may be sent between the prover and verifier before the verifier is convinced the statement is true. Non-interactive (NI) ZKPs require just one message to be sent from the prover, and any verifier can verify a proof upon receipt.

[1] NP consists of the languages for which it is possible to verify membership within polynomial time.

NIZKPs are typically more efficient, and can be performed asynchronously - the prover and verifier need not be active at the same time.

The concept of zero-knowledge was originally introduced in the classic work of S. Goldwasser and C. Rackoff [7] which formalised what it means to gain no knowledge. Much work has been undertaken since then to increase the applicability and efficiency of zero-knowledge for use in practical protocols.

A useful subset of interactive ZKPs are three-message protocols called sigma (or Σ) protocols. Sigma protocols were introduced in the PhD thesis of R. Cramer [8], taking their name from the Σ-shape of the messages between the prover and verifier: the prover sends a commitment, the verifier sends a challenge, and the prover responds to the challenge (see Figure 1.1).

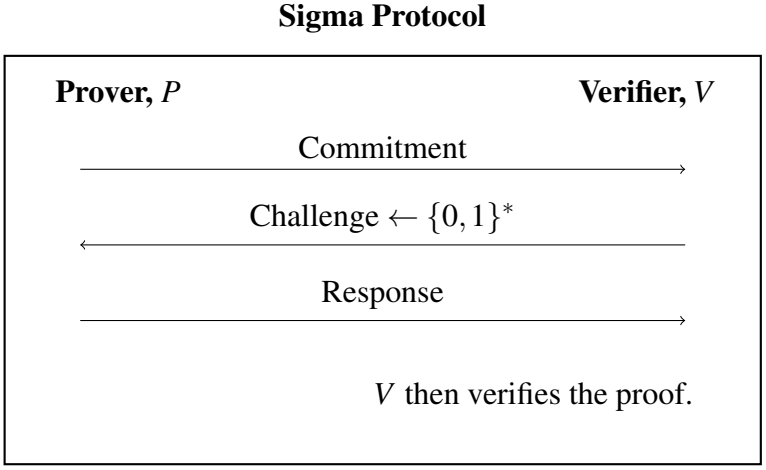

Figure 1.1: Sigma protocols are a 3-message proof system. The prover (P) sends an initial commitment and responds to a randomly generated challenge (from an appropriate challenge space) sent by the verifier (V). The verifier then validates the response and proof.

Sigma protocols are useful for security proofs, enjoying several properties [9]:

- Sigma protocols are invariant under parallel composition, i.e. performing two sigma protocols in parallel is itself a sigma protocol.
- A sigma protocol exists for challenges of any length.
- It is possible to take logical combinations of sigma protocols, for example, the OR Composition, which will be central to this project.

Sigma protocols are only zero-knowledge under the assumption that the verifier acts honestly. This is a strict assumption that is not very applicable in the real world. However, as will be explained in this report, it is possible to transform a sigma protocol into a NI proof. This removes the role of the verifier in producing the proof, hence providing full zero-knowledge.

1.1 Project Goal

Several solutions exist to produce an NI proof from an interactive one. A prevalent and classic example is the Fiat Shamir transform of sigma protocols [10] which replaces the verifier's challenge with randomness generated using a hash function applied to the prover's first message (commitment). Although efficient, the Fiat Shamir transform relies on several unfavourable security properties, which this project addresses.

This project aims to build upon the ideas of [1] and [2] to produce a new NI protocol. Figure 1.2 gives a flowchart indicating the related works, and the setting of this project. The work of M. Fischlin [3] is central to this subject area, from which several alterations and improvements have been made, as indicated in the diagram. Fischlin's insight was to use a hash function in a different way to Fiat Shamir. The transform employs a proof-of-work style approach where the prover is forced to repeatedly generate new proofs until the corresponding transcript of messages hashes to the zero string. While less efficient than Fiat Shamir, this technique enjoys advantageous properties which will be discussed in chapter 3.

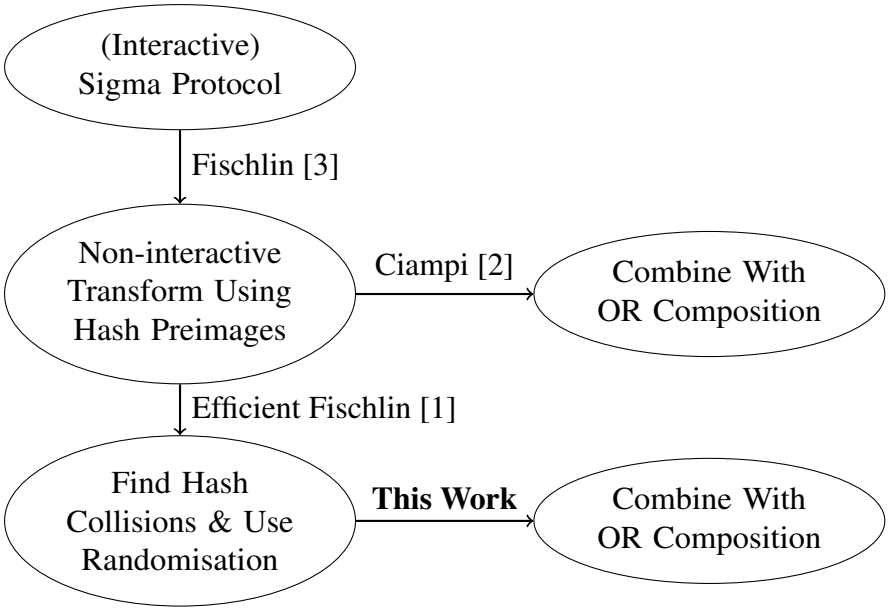

Figure 1.2: Flowchart with related papers and their contributions. The main new concept contributed by each paper is shown within the bubbles.

Fischlin's transform essentially tasks the prover with inverting a hash function. It is possible to improve this approach by instead finding two preimages which share a hash collision, and [1] modifies Fischlin's protocol using this idea, achieving efficiency improvements.

The goal of the new construction in this project is to harness the gains made by [1] and combine two sigma protocols inspired by the OR Composition to produce a NI proof. The aim is to employ the following favourable approaches, which were not all satisfied by Fischlin or Fiat Shamir:

- Prove security in the *non-programmable random oracle model*. Hash functions

are typically modelled as a random function called the random oracle. Many security proofs require the ability to fix certain input-output pairs in the random oracle (this is called programming the random oracle). This is not a realistic approach, so should be avoided when possible.

- Use an *online extractor*. To show the prover knows a secret value (typically called a *witness*), we perform an experiment that *extracts* the witness from the prover. Typical approaches involve rewinding the prover to a previous state to get multiple valid transcripts from which the witness can be computed (an example of this is given in chapter 2). Rewinding the prover causes issues when composing proofs, so it is advantageous to use an online extractor, i.e. one that does not rewind the prover.

Implementations of the protocols involved will also allow empirical comparison of runtime.

1.2 Summary of Contributions

The following is a list of contributions made by this work:

- Interpreting and exploring the definition of *strong r special soundness*, introduced by [1] as a prerequisite property of their transform.
- Construction of a new non-interactive protocol which achieves preferable security properties. A proof of security is also provided.
- Implementation of the protocol (in Python) to compare runtime against Fischlin's protocol [3].

1.3 Report Structure

- Chapter 2 defines zero-knowledge formally and introduces the necessary definitions which are central to the rest of the report.
- Chapter 3 introduces the relevant background literature, exploring existing approaches to non-interactive zero-knowledge and their limitations.
- Chapter 4 provides a new construction and proof of security.

Chapter 2

Preliminaries and Definitions

2.1 Interactive Proof Systems

Before formally defining zero-knowledge proofs, it is useful to consider what a "proof" actually is. Proofs can take several forms, for example, a series of logical steps deriving a mathematical theorem, or a proof in the court of law, or proof of identity at an airport. All these examples have one thing in common: there exists a prover whose role is to convince a verifier of some fact.

To formalise this notion, [7] defined the concept of an Interactive Proof System (IPS) with the goal of defining what an "efficient theorem-proving procedure" must entail.

Definition 2 *[7] An IPS consists of two algorithms P and V, the prover and verifier respectively, and an NP language*[1] *L. P is not computationally bounded, however V must run in Probabilistic Polynomial Time (PPT)*[2]*. We say a statement x is true if $x \in L$ and false if $x \notin L$. Write $(P,V)(x)$ to denote the output of V following the interaction between P and V where both parties are provided with the statement x. V outputs* 1 *or* 0 *depending if it accepts or rejects the proof that $x \in L$ respectively. An IPS must satisfy the following properties:*

- ***Completeness****: The verifier accepts a proof of a true statement with high probability.*

$$\forall x \in L,\ Pr[(P,V)(x) = 1] \geq 1 - \frac{1}{p(|x|)}$$

  for every polynomial p.

- ***Soundness****: The verifier should reject a proof of a false statement with high probability.*

$$\forall x \notin L,\ Pr[(P,V)(x) = 1] \leq \frac{1}{p(|x|)}$$

  for every polynomial p.

[1] A set of statements that can be verified in polynomial time.

[2] A PPT algorithm runs in polynomial time, and can use randomness. Formally, it is a probabilistic Turing machine running in polynomial time.

Remark We can alter the definition to allow an arbitrary soundness error, i.e. $\forall x \notin L,\ Pr[(P,V)(x) = 1] \leq s(|x|)$ for some bound s. However, repeating the protocol multiple times makes the soundness error negligible[3], since it is unlikely a cheating prover can repeatedly guess correctly. It therefore suffices to use the definition above.

The following example illustrates the concept of an IPS.

Example 3 *Consider the language*

$$\begin{aligned} L =& \{x \in SAT\} \\ =& \cup_{n\in\mathbb{N}} \{x : \{0,1\}^n \to \{0,1\} | \exists y = (y_1,...,y_n) \text{ such that } x(y_1,...,y_n) = 1\} \end{aligned}$$

consisting of satisfiable Boolean statements x. Suppose the prover has some arbitrary x and wants to prove to the verifier that $x \in L$. If $x \in L$, then since the prover is computationally unbounded, it can find a satisfying assignment y for x, so the proof proceeds simply by giving y to the verifier, who can efficiently check it is a satisfying assignment. We now need to check the completeness and soundness properties:

- *Completeness: If $x \in L$, then the prover provides y such that y is a satisfying assignment, and the verifier will accept with probability* 1 *since it can simply confirm this is the case.*
- *Soundness: If $x \notin L$, then there does not exist any satisfying assignment y, so whatever y the prover provides will not satisfy x. Hence the verifier will accept with probability* 0*.*

The prover in an IPS is sometimes also given some auxiliary information *aux* that helps it prove a statement or allows it to keep track of previous steps if multiple instances of a prover are used in a proof.

2.2 Simulator Paradigm and Zero-Knowledge

An (interactive) Zero-Knowledge Proof (ZKP) is an IPS such that there exists an algorithm, called the *simulator*, such that whatever messages are produced during the proof for a true statement x can also be outputted by the simulator on input x without access to the prover. In other words, whatever has been learned through the interaction could already have been learned by anyone before the interaction took place. If $x \in L$, V learns nothing new from the interaction, except that $x \in L$. [11] [12]

Definition 4 *[7] A ZKP is an IPS which additionally satisfies the following property:*

- ***(Perfect) Zero-Knowledge**: For all, possibly malicious, PPT verifiers $V^\star$ there exists a PPT algorithm Sim such that the following random variables are identically distributed $\forall x \in L$:*
  - *$view^P_{V^\star}(x)$, a random variable describing the set of messages $V^\star$ receives, (and its own randomness).*
  - *$Sim(x)$, a random variable describing the output of the simulator,*

[3] a function $f(n)$ is negligible if $|f(n)| \leq \frac{1}{p(n)}$ for every polynomial p eventually.

With this definition, it is clear that the IPS in Example 3 is not zero-knowledge, since the verifier learns y when it did not know it before (SAT is not in the complexity class P[4], so V cannot determine y on its own in general, being bounded to polynomial time). The distribution of a simulator's output would be different from the view of the verifier who is provided with y.

Unfortunately, we do not know of any non-trivial case in which perfect zero-knowledge is satisfied by a protocol [12]. Relaxing the zero-knowledge condition so that the two probability distributions are computationally indistinguishable gives rise to computational zero-knowledge.

Definition 5 *[12] A distinguisher is an algorithm that outputs* 0 *or* 1 *depending on whether it believes it received an input from one of two possible classes. Two ensembles* $\{A(x)\}_{x\in L}$ *and* $\{B(x)\}_{x\in L}$ *are computationally indistinguishable if for every PPT distinguisher D, and every polynomial p, it holds that*

$$|Pr[D(x,A(x)) = 1] - Pr[D(x,B(x)) = 1]]| < \frac{1}{p(|x|)}.$$

Definition 6 *[12] An IPS has computational zero-knowledge if it satisfies the following property:*

- ***(Computational) Zero-Knowledge:*** *For all, possibly malicious, PPT verifiers* $V^\star$ *there exists a PPT algorithm Sim such that the following two ensembles are computationally indistinguishable:*
  - $\{(P,V)(x)\}_{x\in L}$
  - $\{Sim(x)\}_{x\in L}$

2.3 Proofs of Knowledge

Zero-knowledge captures the notion of learning nothing new from an interaction, but it is also important to ensure the prover actually *knows* the secret information used to produce the proof. For example, in key distribution protocols, it might be useful to prove *knowledge* of a shared key, without revealing it (for example [13]).

The concept of (Zero-Knowledge) Proofs of Knowledge ((ZK) PoKs) captures this idea by introducing the concept of a witness, w, to a common input x. A witness is a secret value known only to the Prover that is somehow related to x and the fact that $x \in L$. In Example 3, the witness would be the satisfying assignment y. The goal of a proof of knowledge is to show that the prover actually knows the witness w.

Definition 7 *[12] A relation* Rel_L *over a language L is defined to be the set of pairs of statements and their witnesses:*

$$Rel_L := \{(x,w) : x \in L \wedge w \text{ is a witness for } x\}.$$

[4] P consists of problems solvable in polynomial time.

To formalise PoKs, the concept of an *Extractor* was introduced. An Extractor is an algorithm E which, given an x such that $(x,w) \in Rel_L$ for some w, outputs a witness w for x with high probability. If the extractor can do so by interacting only with the prover P, then this suggests that P indeed knows w. The extractor can also *rewind P* to a previous state. This allows the extractor to try different responses (which would not be possible in an ordinary proof), allowing it to compute a witness.

Definition 8 *[12] [14] A ZKPoK over a language L is a protocol between a prover P and a verifier V such that the following conditions hold:*

1. ***Completeness**: The verifier accepts a proof of a true statement for which P has a witness with high probability. For every $(x,w) \in Rel_L$,*

$$Pr[(P(w),V)(x) = 1] \geq 1 - \frac{1}{p(|x|)}$$

for every polynomial p.

2. ***Soundness / Extractor**: There exists an extractor E such that for every $(x,w) \in Rel_L$, the extractor outputs a witness w' such that $(x,w') \in Rel_L$ with high probability.*

   Specifically, for all provers $P^\star$, the probability E produces a valid witness w' while interacting with prover $P^\star$ is at least as high as the probability V accepts while interacting with $P^\star$, up to some error ε.

3. ***(Computational) Zero-Knowledge**: For all, possibly malicious, PPT verifiers $V^\star$ there exists a PPT algorithm Sim such that the following two ensembles are computationally indistinguishable:*

   - $\{(P(w),V)(x)\}_{x\in L}$
   - $\{Sim(x)\}_{x\in L}$

A PoK can also be defined without zero-knowledge by removing the zero-knowledge requirement in the definition.

The following protocol due to Schnorr [15] illustrates the concepts of simulators and extractors.

Example 9 *The Schnorr protocol relies on the hardness of the discrete logarithm problem, i.e. when provided with a term g^w for some group generator g, it is hard to determine the exponent w.*

Schnorr Protocol [15]

Let G be some group of prime order q, and let g be a generator for the group. This information is publicly known. The prover P and verifier V receive the common input $x = g^w$, where w is a secret witness for x known only by P. P's goal is to prove knowledge of w.

1. *P picks a random $r \leftarrow \mathbb{Z}_q$ uniformly, and sends $a := g^r$ to V.*

2. *Upon receiving a, V sends a random challenge* $e \leftarrow \mathbb{Z}_q$*, drawn uniformly, back to P.*
3. *P then responds by sending* $z := r + ew$ *to V.*
4. *V checks whether* $g^z = ax^e$ *and outputs* $1/0$ *depending on whether this is the case.*

Schnorr's protocol is a PoK. It satisfies:

1. *Completeness: Suppose* $(x,w) \in Rel = \{(x,w) : x = g^w\}$*. Then proceeding as in the protocol,* $g^z = g^{r+ew} = g^r \cdot (g^w)^e = g^r \cdot x^e = ax^e$*, so the verifier outputs* 1 *with probability* 1*.*
2. *Soundness/Extractor: Construct an extractor E as follows:*

   Schnorr Extractor

   (a) Run P to obtain a.

   (b) Send a random $e_1 \leftarrow \mathbb{Z}_q$ *to P and store the response* z_1 *received from P.*

   (c) Rewind P and repeat by sending a new different random $e_2 \leftarrow \mathbb{Z}_q$*. Store the response* z_2 *received from P.*

   (d) Output the quantity $w' = \frac{z_1 - z_2}{e_1 - e_2}$*.*

   Assume $(x,w) \in Rel$*. We know* $g^{z_1} = ax^{e_1}$ *and* $g^{z_2} = ax^{e_2}$*, so* $a = \frac{g^{z_1}}{x^{e_1}} = \frac{g^{z_2}}{x^{e_2}}$*, so* $g^{z_1 - z_2} = x^{e_1 - e_2}$*. We also know* $x = g^w$*, so* $(g^w)^{e_1 - e_2} = g^{z_1 - z_2}$ *and hence* $x = g^w = g^{\frac{z_1 - z_2}{e_1 - e_2}}$*, and hence* $w' = \frac{z_1 - z_2}{e_1 - e_2}$ *is a valid witness.*
3. *Zero-Knowledge: Construct a simulator Sim as follows:*

   Schnorr Simulator

   (a) Pick a random $z \leftarrow \mathbb{Z}_q$ *uniformly.*

   (b) Pick a random $e \leftarrow \mathbb{Z}_q$ *uniformly.*

   (c) Set $a = \frac{g^z}{x^e}$*.*

   (d) Output (a,e,z)*.*

   Since z is chosen randomly, the message a will also be random, so the distribution of the output of the simulator will be identical to the distribution of the view of the verifier in the protocol. This is technically a proof of "Honest Verifier" zero-knowledge, see section 2.4 on Sigma Protocols for more details.

2.4 Sigma Protocols

Schnorr's protocol (Example 9) is a textbook example of a type of protocol known as a *sigma protocol*, or Σ *protocol*. This is also found in the first bubble of Figure 1.2, and is

a central topic throughout the rest of this project.

Definition 10 *[8] Sigma protocols take a 3-message form, consisting of an initial commitment a from the prover P, a random challenge e from the verifier V, and a response z from P (see Figure 1.1).*

All sigma protocols (for a relation Rel) must satisfy the following conditions:

- ***Completeness**: If $(x,w) \in Rel$ where x is the common input to P and V, and w is a private input to P, then V always accepts if the protocol is followed honestly.*
- ***Special Soundness**: There exists an efficient extractor E which given **two** accepting transcripts $(a,e,z),(a,e',z')$ for the same initial commitment, and $e \neq e'$, can output a witness w' such that $(x,w') \in Rel$.*
- ***Special Honest Verifier Zero-Knowledge (SHVZK)**: There exists an efficient simulator Sim which upon input x and a challenge e, outputs a and z such that (a,e,z) is an accepting transcript. If e is chosen uniformly, then (a,e,z) is distributed identically to a real transcript.*

Sigma protocols are a method of obtaining efficient zero-knowledge. SHVZK and special soundness makes proofs easier by allowing easier construction of simulators and extractors. Sigma protocols are also proofs of knowledge; this is shown in [9].

The SHVZK requirement essentially says that if the verifier follows the protocol honestly (i.e. sends a uniformly distributed random challenge), then the zero-knowledge requirement from before is satisfied. (The term "Special" refers to the fact that the simulator is provided with a challenge *e* beforehand. Some authors drop this requirement, and the property is then simply called *Honest Verifier Zero-Knowledge*).

SHVZK is a weaker property than ZK, since we have to assume the verifier follows the protocol honestly. This is not always the case in reality; however, solutions exist. For example, the GMW compiler for multiparty communication [6] assumes the parties act honestly and instead ensure security by forcing them to prove in zero-knowledge that they acted honestly.

It is also possible to turn a sigma protocol into a non-interactive protocol (this will be discussed later), and this transforms SHVZK back to full ZK.

2.5 The OR Composition & Witness Indistinguishability

Sigma protocols are particularly useful since it is possible to take logical combinations of them. A popular transform is the *OR Composition* which allows a prover *P* to prove that they know a witness *w* for one of two possible inputs x_0, x_1 belonging to two sigma protocols Σ_0, Σ_1 respectively, but without revealing to which. *P* achieves this by utilising the SHVZK simulator for the sigma protocol for which they do not know the witness.

The OR Composition protocol is given below[5].

[5] $\oplus$ is used to indicate the logical XOR operation, or addition modulo 2.

OR Composition [9]

Let $\Sigma_0 = (\mathcal{P}_0, \mathcal{V}_0)$ and $\Sigma_1 = (\mathcal{P}_1, \mathcal{V}_1)$ be two sigma protocols for the relations Rel_0, Rel_1 respectively. Construct a new protocol consisting of a prover P and a verifier V as follows:

1. P and V receive a common input $x = (x_0, x_1)$ and P receives a witness w such that $(x_0, w) \in Rel_0 \vee (x_1, w) \in Rel_1$. Let $b \in \{0,1\}$ denote whether w is a witness for x_0 or for x_1 respectively.
2. P picks a random $e_{1-b} \leftarrow \{0,1\}^l$, where l is the predefined challenge length of Σ_0 and Σ_1. P then uses the SHVZK simulator Sim_{1-b} of Σ_{1-b} to obtain $(a_{1-b}, z_{1-b}) \leftarrow Sim_{1-b}(x_{1-b}, e_{1-b})$.
   P now has a proof transcript $(a_{1-b}, e_{1-b}, z_{1-b})$ for Σ_{1-b}.
3. P uses $\mathcal{P}_b$ to generate a commitment $(a_b, aux_b) \leftarrow \mathcal{P}_b(x_b, w)$. The aux_b is an auxiliary term used to keep track of the state of $\mathcal{P}_b$ at this moment.
4. P sends a message $\boldsymbol{a} := (a_0, a_1)$ to V.
5. V, upon receipt of $\boldsymbol{a}$, sends back a random challenge $\boldsymbol{e} \leftarrow \{0,1\}^l$ drawn uniformly.
6. P, upon receipt of $\boldsymbol{e}$, computes $e_b := \boldsymbol{e} \oplus e_{1-b}$. P then uses e_b and aux_b to obtain $z_b \leftarrow \mathcal{P}_b(aux_b, e_b)$ from the prover $\mathcal{P}_b$.
   P now has a proof transcript (a_b, e_b, z_b) for Σ_b.
7. P sends $\boldsymbol{z} := (e_0, z_0, e_1, z_1)$ to V.
8. V parses $\boldsymbol{z}$ and checks that $\mathcal{V}_0(a_0, e_0, z_0) = \mathcal{V}_1(a_1, e_1, z_1) = 1$ and that $\boldsymbol{e} = e_0 \oplus e_1$. If these checks are satisfied, V outputs 1, else 0.

It turns out that the OR composition of two sigma protocols is also a sigma protocol in its own right.

Theorem 11 *[9] The OR composition of Σ_0 and Σ_1 is a sigma protocol for the relation $Rel_{OR} := \{((x_0, x_1), w) : (x_0, w) \in Rel_0 \vee (x_1, w) \in Rel_1\}$. That is, it satisfies completeness, special soundness, and special honest verifier zero-knowledge.*

The proof of Theorem 11 is given in [9]. It is easy to see completeness; the **bold** messages $\boldsymbol{a}, \boldsymbol{e}, \boldsymbol{z}$ give the required three-message form. Observe also that the two challenges e_b and e_{1-b} are valid according to the protocol: e_{1-b} is generated uniformly randomly, and e_b is produced by taking the one-time pad [16] with another uniformly random string e, so it will also be uniformly random.

The OR composition also enjoys a property called *Witness Indistinguishability* (WI). This is a weaker property than ZK that allows a prover to hide which witness was used to produce the proof.

Definition 12 ***Witness Indistinguishability (WI) [17]*** *Consider the experiment E^b_{WI} consisting of two players, a challenger Ch and adversary A, defined as follows.*

The adversary A picks an x and two valid witnesses w_0, w_1 for x such that $(x, w_0) \in Rel$

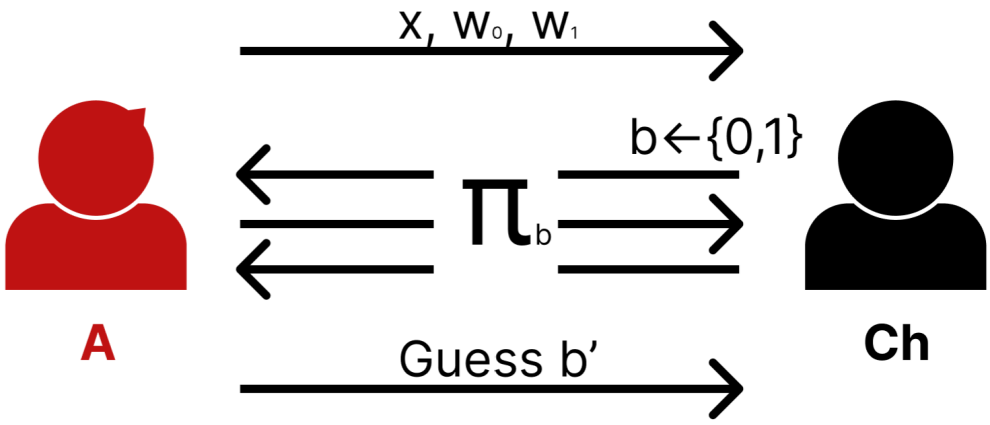

Figure 2.1: Witness Indistinguishability Game

and $(x, w_1) \in Rel$. *Ch picks a random* $b \in \{0,1\}$. *Let* Π *be a proof system (e.g. sigma protocol) that we want to prove has the WI property. Ch interacts with A following the prover procedure P of* Π, *using the witness* w_b. *A then outputs a guess* b' *for b and wins the experiment if it can guess correctly. The experiment is shown graphically in Figure 2.1.*

We then say Π *is witness indistinguishable if for every distinguisher D (recall Definition 5), the experiments* $E_{WI}^{b=0}$ *and* $E_{WI}^{b=1}$ *are computationally indistinguishable.*

[9] and [18] provide proofs that the OR Composition of two sigma protocols is Witness Indistinguishable. So not only can a prover hide which witness they know for one of two possible inputs x_0, x_1, they can also hide which witness they know given two possible candidates.

One reason we are interested in whether a protocol is witness indistinguishable is that the WI property holds when we compose several WI protocols in parallel, i.e. sending all first messages, then all second messages, and so on. The same cannot be said for ZK. [19] provides a proof that (computational) ZK does not hold under parallel composition. This is a limitation that ZKPs bear; composing proofs in parallel produces an efficiency advantage for use in protocols, however we cannot directly apply this to ZKPs. If full ZK is not required and WI suffices, this is no longer an issue.

It is also worth remarking that, as we would expect, ZK implies WI. This follows since by definition the ZK simulator's outputs must be (computationally) indistinguishable from the verifier's view in a real interaction, independent of which witness was used. [20] gives a full proof sketch. For a simple counter example to the converse statement, consider a relation *Rel* where each x has exactly one witness w_x. Consider a protocol in which the prover sends the witness w_x to the verifier (as was the case in Example 3). This is clearly not ZK, but satisfies the WI definition (the same witness w_x would be provided for $b = 0$ and $b = 1$).

Chapter 3

Existing Approaches

Chapter 2 introduced IPSs and variants such as sigma protocols and the OR composition. These protocols enjoy many properties like zero-knowledge and witness indistinguishability, however, they all have one major drawback: they are interactive. By nature, interactive proofs are quite inefficient in practice, since it takes time to send, receive, and process messages in the real world. The solution is to remove interaction altogether.

Definition 13 ***Adapted from [12]*** *A Non-Interactive (NI) Zero-Knowledge proof (NIZK proof) is a pair of probabilistic algorithms (P,V), the prover and verifier respectively, such that V runs in polynomial time. P,V receive the common input x, and P produces a proof π for x which V accepts or rejects. The following properties must also be satisfied:*

- ***Completeness****: For every $x \in L$, V accepts the proof $\pi \leftarrow P(x)$ with high probability.*
- ***Soundness****: For every $x \notin L$, V rejects the proof $\pi \leftarrow P(x)$ with high probability for every possibly malicious P.*
- ***(Computational) Zero-Knowledge****: There exists a simulator Sim such that the following ensembles are computationally indistinguishable:*
  - $\{P(x)\}_{x \in L}$
  - $\{Sim(x)\}_{x \in L}$

The definition can be adapted to work for proofs of knowledge as before in section 2.3 with the introduction of witnesses and an extractor.

3.1 Fiat Shamir Transform

The Fiat Shamir transform (FS transform) [10] is a popular method of producing a NI proof from an interactive one. The FS transform takes as input a three-message (a,e,z) sigma protocol Σ. Recall this means that Σ satisfies completeness, special soundness, and SHVZK.

In a sigma protocol, the verifier V's only role (other than verifying) is to provide a

random string e. The FS transform instead lets the prover P generate randomness itself, removing the role of V. P cannot simply generate a random string like V does since this would allow a cheating prover to produce a valid proof by repeatedly trying challenges until finding one that makes them appear honest.

The trick is to make the random challenge generated by P deterministic based on the previous message a and the common input x using a hash function $H : \{0,1\}^* \to \{0,1\}^l$. This way a cheating prover cannot try new challenges.

The FS Transform is summarised below.

Fiat Shamir Transform [10]

Let $\Sigma = (P,V)$ be a sigma protocol for language L with common input x. Let w be a witness for x. Let H be a hash function with fixed output dimension.

The FS Prover:

1. Does nothing to the commitment; sets $(\boldsymbol{a}, aux) \leftarrow P(x,w)$.
2. Sets $\boldsymbol{e} := H(x,\boldsymbol{a})$.
3. Computes $\boldsymbol{z} := P(aux,\boldsymbol{a},\boldsymbol{e})$.
4. Outputs the proof $(\boldsymbol{a},\boldsymbol{e},\boldsymbol{z})$, which is then public.

Any verifier can then check the validity of the proof using V.

The goal of a hash function is to assign a random string of finite length to an input of arbitrary length. However, in reality, hash functions are pseudorandom since there is no known way of generating true randomness with classical computers. Despite this, hash functions are often modelled as a *random oracle* in cryptography [21]. This allows for much simpler security proofs.

Definition 14 ***(discussed in [21])*** *A random oracle O is a random function of fixed output length. O is deterministic in the sense that providing the same input x twice yields the same result $O(x)$, and $O(x)$ is random with no correlation to x.*

The FS transform is very efficient and simple, making use of a single hash function application to remove interaction. Converting to a NI proof like this also allows us to remove the "honest-verifier" requirement of ZK in sigma protocols since we have removed the role of the verifier entirely.

However, the proofs of security [22] of the FS transform treats H as a random oracle which must be *programmed* by the simulator, i.e. the simulator decides the outputs for specific messages in the oracle while ensuring the overall output distribution is still uniformly random. It would be better to avoid programming the random oracle, as this better matches the real world. Additionally, the extractor also requires the ability to rewind the prover, which poses issues for composing proofs (the following sections discuss protocols that do not use a rewinding extractor). Rewinding the prover also sacrifices "tightness" of the security reduction [1]; if the input protocol Σ has extraction error κ then the Fiat Shamir transform can have an extraction error as bad as $Q^n \cdot \kappa$ where Q is the number of queries the adversary makes to the random oracle, and n is

the number of challenges sent by the prover during rewinding [23].

Another limitation of the FS transform is that, according to [24] and [25], there are protocols such that when the FS transform is applied, no implementation of the hash function H produces a secure scheme. Hence it is impossible to guarantee security of Fiat Shamir without the random oracle model.

3.2 Fischlin's Transform

Fischlin [3] developed the idea of using a hash function from the FS Transform by introducing a proof-of-work style protocol. The goal of his work was to create a NI PoK that required only an *online* extractor, which is an extractor that does not need to rewind the prover to produce a witness. Protocols with online extractors do, however, need setup assumptions, such as the random oracle [1].

Most protocols with online extractors build on sigma protocols by forcing the prover to compute at least two proof transcripts with the same first message a with extremely high probability (extraction fails with the remaining negligible probability). The extractor can then use properties of the underlying sigma protocol to compute the witness.

Roughly speaking, to allow online extraction, the prover in Fischlin's protocol must compute a transcript π whose hash evaluates to the zero string 0^l. It does this by trying different challenges e. This process is then repeated r times to increase soundness. The full protocol is given below.

Fischlin's NI PoK Transformation [3]

Let $\Sigma = (\mathcal{P}, \mathcal{V})$ be a sigma protocol (with quasi-unique third round z). Let P,V be the new prover and verifier, who receive x as common input. P also receives a witness w for x. Let λ be the security parameter.[a] Pick integers r,l,t such that $r \cdot l = \lambda$ and $t = \lceil \log_2 \lambda \rceil \cdot l$, where r is the number of repetitions, l is the random oracle O's output length, and t is the maximum number of tries on a given repetition.

The prover $P(x,w)$:

1. Computes r commitments $(a_i, aux_i) \leftarrow \mathcal{P}(x,w)$ for $i \in [r]$ and sets $\boldsymbol{a} := (a_0, \cdots, a_r)$.
2. Initialises $e_i = -1$ for $i \in [r]$.
3. For each $i \in [r]$:
   (a) Aborts the proof if $e_i > t$ (too many tries).
   (b) Increments $e_i = e_i + 1$ and computes $z_i \leftarrow \mathcal{P}(aux_i, e_i)$.
   (c) Repeats 3a and 3b until $O(\boldsymbol{a}, i, e_i, z_i) = 0^l$ (or aborts).
4. Sets $\boldsymbol{e} := (e_1, \cdots, e_r)$, $\boldsymbol{z} := (z_1, \cdots, z_r)$ and outputs the proof $\pi := (\boldsymbol{a}, \boldsymbol{e}, \boldsymbol{z})$.

Any verifier $V(x)$ on receipt of a proof π:

1. Parses $(a_i, e_i, z_i)_{i\in[r]}$ from π.
2. For each $i \in [r]$, verifies that $O(\boldsymbol{a}, i, e_i, z_i) = 0^l$ and that $\mathcal{V}(x, a_i, e_i, z_i) = 1$.
3. Outputs 1 if both checks are accepting, else 0.

[a]The security parameter λ controls the difficulty of breaking the scheme for an attacker.

Online extraction is then simple. The random oracle output will be 0^l on the first try with probability only $\frac{1}{2^l}$ since there are 2^l possible (random) strings of length l. Hence it is very likely that P will make at least two queries to the random oracle, with the same a but different challenges e_i. Hence, equipped with two valid transcripts for Σ, the online extractor can appeal to the special soundness of Σ to produce a valid witness w for x.

Fischlin's protocol can only be applied to sigma protocols with quasi unique responses, i.e. the probability two transcripts with different responses for the same challenge are accepting is negligible. This requirement was used in Fischlin's proof. Unfortunately, the OR Protocol does not have quasi-unique responses, so Fischlin's protocol cannot be applied directly. Randomisation fixes the problem (this will be explained in later sections).

3.3 Ciampi's Transform

Unfortunately, it is impossible to produce a NIZK proof for non-trivial languages without any setup assumptions such as initial shared randomness (called a *Common Reference String (CRS)*, proposed by Blum [26]) between the prover and verifier. While it is possible to establish a CRS between parties, this makes protocols less efficient and often assumes the existence of a trusted third party.

To see the impossibility of NIZK without a CRS, assume there was a NIZK proof for some x. Then the ZK simulator *Sim* can produce a valid proof transcript π for x. A cheating verifier can therefore call $Sim(x)$ to get a transcript π', and then use the real verifier on input π' as a subroutine. If the real verifier outputs 1 then the cheating verifier knows $x \in L$, and if it outputs 0 then $x \notin L$. Hence the cheating verifier can determine whether $x \in L$ without even interacting with the prover. Hence, the only NIZK proofs are for those languages L such that it is trivial (or efficient) to check membership.

It is still possible to guarantee WI in an NI setting without setup assumptions like a CRS. Ciampi's transform [2] combines a modified version of Fischlin's transform (section 3.2) with the OR composition (section 2.5) to produce a NI WI PoK. Using the OR composition allows WI, and Fischlin's transform allows for online extraction. Ciampi's proof of security also holds in the non-programmable random oracle model, which is an advantage over the original FS transform.

For completeness, the protocol is given below.

Ciampi's Transform [2]

Let $\Sigma_0 = (\mathcal{P}_0, \mathcal{V}_0), \Sigma_1 = (\mathcal{P}_1, \mathcal{V}_1)$ be two sigma protocols, with challenge length

k. Let λ be the security parameter, and define integers l, r, t such that $l \cdot r = \omega(\log\lambda), 2^{t-l} = \omega(\log\lambda)$ and $l, r, t = O(\log\lambda)$ with $l \leq t \leq k$. Let O be a random oracle that maps to l bits. Denote a new prover and verifier P, V respectively.

Inputs: Common input $x := (x_0, x_1)$ (and parameters above) and private input w_b to P such that w_b is a witness for x_b. ($b \in \{0,1\}$).

Proof, by P:

1. Run $\mathcal{P}_b$ r times to obtain $(a_b^{(1)}, aux_b^{(1)}) \leftarrow \mathcal{P}_b(x_b, w_b), \cdots, (a_b^{(r)}, aux_b^{(r)}) \leftarrow \mathcal{P}_b(x_b, w_b)$, using fresh randomness in $\mathcal{P}_b$ each time.
2. For $i \in [r]$ pick random challenges $e_{1-b}^{(i)} \leftarrow \{0,1\}$ uniformly, and use the SHVZK simulator of Σ_{1-b} to compute $(a_{1-b}^{(i)}, z_{1-b}^{(i)}) \leftarrow Sim_{1-b}(x_{1-b}, e_{1-b}^{(i)})$ for each i.
3. Set $\boldsymbol{a} := ((a_0^{(0)}, a_1^{(0)}), ..., (a_0^{(r)}, a_1^{(r)}))$.
4. For each $i \in [r]$:
   (a) Define and set $\mathcal{E}_i := \emptyset$.
   (b) Sample a new random $\boldsymbol{e}^{(i)} \leftarrow \{0,1\}^t \setminus \mathcal{E}_i$ and compute $e_b^{(i)} := e_{1-b}^{(i)} \oplus \boldsymbol{e}^{(i)}$. Compute $z_b^{(i)} \leftarrow \mathcal{P}_b(aux_b^{(i)}, e_b^{(i)})$.
   (c) If $O(x_0, x_1, \boldsymbol{a}, i, \boldsymbol{e}^{(i)}, e_0^{(i)}, e_1^{(i)}, z_0^{(i)}, z_1^{(i)}) \neq 0^l$ then add $\boldsymbol{e}^{(i)}$ to $\mathcal{E}_i$ and go back to step 4b.
5. Output the proof $\pi := (\{a_0^{(i)}, a_1^{(i)}, \boldsymbol{e}^{(i)}, e_0^{(i)}, e_1^{(i)}, z_0^{(i)}, z_1^{(i)}\}_{i \in [r]})$.

Verification, by V**:**

1. Parse π upon receipt.
2. For each $i \in [r]$:
   (a) Check $\mathcal{V}_0(x_0, a_0^{(i)}, e_0^{(i)}, z_0^{(i)}) = 1$ and $\mathcal{V}_1(x_1, a_1^{(i)}, e_1^{(i)}, z_1^{(i)}) = 1$.
   (b) Check $\boldsymbol{e}^{(i)} = e_0^{(i)} \oplus e_1^{(i)}$.
   (c) Check $O(x_0, x_1, \boldsymbol{a}, i, \boldsymbol{e}^{(i)}, e_0^{(i)}, e_1^{(i)}, z_0^{(i)}, z_1^{(i)}) = 0^l$.
3. If all checks are satisfied, output 1, else 0.

3.4 Efficient Fischlin

The new construction in this project takes a similar approach to Ciampi [2]. Instead of building upon Fischlin's construction, a modified, more efficient, version is used. In Figure 1.2 this is the second vertical arrow named "Efficient Fischlin" and represents the work of Kondi and Shelat [1].

Efficient Fischlin improves Fischlin's protocol [3] in two key areas:

- **Randomisation.** Fischlin's protocol suffers from determinism. After the commitments $\boldsymbol{a}$ have been sampled by the prover, the rest of the protocol is deterministic. [1] shows that this leads to an attack on witness indistinguishability when a language with multiple witnesses is used. Instead of picking a new challenge e_i by incrementing, a new random challenge is sampled each time in the Efficient Fischlin protocol, thus removing determinism.
- **Hash Collisions.** Fischlin's protocol essentially forces the prover to find a *hash preimage* of the zero string (r times). Efficient Fischlin reduces oracle query complexity by instead forcing the prover to find (r) *hash collisions*, which is more probable than finding a preimage. This reason for this is discussed later in section 4.8.1.

The Efficient Fischlin protocol takes a special kind of sigma protocol as input. If r collisions are required in the protocol, then an $r+1$ *strong special sound* and r *simulatable* sigma protocol is required.

Definition 15 *[1] A three-move protocol Σ is an $r+1$ strong special sound sigma protocol if it satisfies the following properties:*

- ***Completeness:*** *as usual.*
- ***(r+1) Strong Special Soundness:*** *There exists an efficient extractor E which given as input $r+1$ accepting transcripts $\{\pi_i := (a, e^{(i)}, z^{(i)})\}_{i\in[r]}$ for a statement x, such that $\pi_i \neq \pi_j \ \forall i, j \in [r]$, can output a witness w for x.*
- ***HVZK With r-Simulatability:*** *There exists an efficient simulator Sim which upon input x and r challenges $\{e^{(i)}\}_{i\in[r]}$ outputs $(a, \{z^{(i)}\}_{i\in[r]})$ such that each $(a, e^{(i)}, z^{(i)})$ is an accepting conversation.*

Strong special soundness differs from special soundness since it requires transcripts π_i, π_j to be different rather than only challenges $e^{(i)} \neq e^{(j)}$. This means that the extractor can produce a proof if the challenges are the same, so long as the responses $z^{(i)}$ are different.

The efficient protocol is given below.

Efficient Fischlin Transform [1]

Let $\Sigma = (\mathcal{P}, \mathcal{V})$ be an $r+1$ special sound and r simulatable sigma protocol. Let O be a random oracle of output length l, and define $t := l + \lceil \log r \rceil$. Define a new prover P and verifier V. Both receive the common input x and P additionally receives a witness w for x.

The prover $P(x, w)$:

1. Obtains $(\boldsymbol{a}, aux) \leftarrow \mathcal{P}(x, w)$.
2. Defines and sets $\mathcal{E}, \mathcal{Z} = \emptyset$ and repeats:
   (a) Uniformly samples a new $e \leftarrow \{0,1\}^t \setminus \mathcal{E}$.

(b) Obtains $z = \mathcal{P}(aux, e)$ and appends e to $\mathcal{E}$ and (e, z) to $\mathcal{Z}$.

(c) If there exist r pairs

$$(e^{(1)}, z^{(1)}), \cdots, (e^{(r)}, z^{(r)}) \in \mathcal{Z}$$

such that

$$O(\boldsymbol{a}, e^{(1)}, z^{(1)}) = \cdots = O(\boldsymbol{a}, e^{(r)}, z^{(r)})$$

then set $\boldsymbol{e} := (e^{(i)})_{i\in[r]}$ and $\boldsymbol{z} := (z^{(i)})_{i\in[r]}$ and output the proof $\pi := (\boldsymbol{a}, \boldsymbol{e}, \boldsymbol{z})$.

The verifier $V(x)$ upon receipt of a proof π then:

1. Parses $\boldsymbol{a}$, $\boldsymbol{e} = (e^{(i)})_{i\in[r]}$ and $\boldsymbol{z} = (z^{(i)})_{i\in[r]}$.
2. Checks

$$O(\boldsymbol{a}, e^{(1)}, z^{(1)}) = \cdots = O(\boldsymbol{a}, e^{(r)}, z^{(r)}).$$

3. For each $i \in [r]$ checks that $\mathcal{V}(x, \boldsymbol{a}, e^{(i)}, z^{(i)}) = 1$.
4. Outputs 1 if the above checks are satisfied, else 0.

It is worth remarking why $r+1$ strong special soundness and r simulatability is required to produce a secure scheme. The verifier is provided with r accepting transcripts for the same commitment $\boldsymbol{a}$ in an ordinary execution of the protocol. If we only required special soundness (i.e. two required transcripts), then a cheating verifier could use the extractor on input any two of the r accepting transcripts to produce a valid witness w, hence breaking zero-knowledge. This issue is prevented by requiring $r+1$ accepting transcripts. The simulator should also be able to (efficiently) simulate r transcripts for the same commitment a to ensure indistinguishability from a true protocol execution.

Chapter 4

A New Non-Interactive Proof

In this chapter, a new transform is proposed for producing a WI NIPoK with a non-programmable random oracle and online extractor. The efficiency advantages of Efficient Fischlin are combined with the WI property of the OR composition of sigma protocols. These concepts were introduced in Chapters 2 and 3.

4.1 A Word on Notation

In the following chapters, the notation in some of the protocols can get quite involved, so it is worth stepping aside to clarify this.

- The left arrow $\leftarrow$ is used for two purposes which should be clear from context: to denote uniform random sampling from a set, and to denote storing the output of some algorithm.
- When two protocols are provided as input, they are distinguished with the labels "0" and "1". A *subscript* label $b \in \{0,1\}$ is used to keep track of this. This is also used to keep track which variables are inputs/outputs to which protocol.
- When several copies of a variable are stored, the instance is indicated with a *superscript* index (i).
- In the context of sigma protocols:
  - Commitments are indicated using an a.
  - Challenges are indicated using an e.
  - Responses are indicated using a z.
- In the context of sigma protocols, the prover P and verifier V can be called multiple times, representing different stages in the protocol. For example, P may be called on input (x,w) resulting in the output of a commitment a (and auxiliary information *aux* which keeps track of the state of the prover). It may also be called on input (aux,e) for some challenge e, in which case it will return a

response z. The same name P is used in both instances and the output type should be clear from context.

4.2 Preliminary Results Exploring Strong Special Soundness

The concept of strong special soundness was introduced in Chapter 3. This section explores this property to gain a better intuition of its behaviour since $r+1$ strong special sound and r simulatable sigma protocols will also be assumed in the construction in this chapter.

It would be interesting to investigate what kind of sigma protocols satisfy 2 strong special soundness (i.e. $r+1$ strong special soundness with $r=1$). The following theorem says that, rather than having (quasi) unique responses which was required in Fischlin's protocol, if the response is computed deterministically based on the challenge, then strong special soundness holds.

Theorem 16 *Let Σ be a sigma protocol and (a,e,z) be any accepting transcript. If z is computed deterministically given e then Σ is also (2-) strong special sound.*

Proof: Let $(a,e,z),(a,e',z')$ be two different accepting transcripts. We examine two cases. First, suppose $e \neq e'$. Then a witness can be found by special soundness of Σ. Now, suppose $z \neq z'$. Since z,z' are computed deterministically given e,e' respectively, we cannot have $e = e'$. Hence, we can again invoke special soundness of Σ to produce a witness.

Schnorr's protocol (recall example 9) is an example of a protocol on which the ideas of this theorem can be applied. The protocol is a sigma protocol so, in particular, two accepting transcripts $(a,e,z),(a,e',z')$ for the same a allows extraction of a witness. Recall that the prover's initial commitment is $a = g^r$ (r is a random string, not r in the context of r special sound) and the response is computed using $z = r + ew$. The generator g is for a group of prime order, so there is no $r' \neq r \in \mathbb{Z}_q$ ($q \leq p$ prime) such that $g^r = g^{r'} = a$. Hence z is calculated deterministically given e and a in these two transcripts, so Schnorr is strong special sound by the proof of the above theorem.

It is also easy to see the following theorem.

Theorem 17 *If Σ is a strong special sound sigma protocol, then it also satisfies $r+1$ strong special soundness. (Note, this does not say Σ is an $r+1$ strong special sound sigma protocol, since it may not satisfy r simulatability).*

Proof: Suppose we have $r+1$ different such accepting transcripts $(a,e^{(1)},z^{(1)}),\ldots,$ $(a,e^{(r+1)},z^{(r+1)})$. The extractor can pick any two of these transcripts and pass them as input to the extractor for strong special soundness, which exists by assumption. The strong special soundness extractor will generate a valid witness.

The new construction uses ideas from the OR composition of sigma protocols, so it would be interesting to determine whether using two r strong special sound sigma

protocols as input to the OR composition still yields an r strong special sound sigma protocol. It turns out that this is indeed the case.

Theorem 18 *Let Σ_0, Σ_1 be two r strong special sound sigma protocols. Then the OR composition of Σ_0 and Σ_1 is also an r strong special sound sigma protocol.*

Proof:

- *(Completeness): follows since [9] shows the OR composition outputs an ordinary sigma protocol, which satisfies completeness.*
- *(r strong special soundness): Given r accepting transcripts $(a_0, a_1, e_0, e_1, z_0, z_1)^{(i)}$ for $i \in [r]$ send $\{(a_0, e_0, z_0)^{(i)}\}_{i\in[r]}$ to the extractor for Σ_0 or $\{(a_1, e_1, z_1)^{(i)}\}_{i\in[r]}$ to the extractor for Σ_1. Both will return a valid witness.*
- *($r-1$ simulatability): The simulator gets input $x = (x_0, x_1)$ and $r-1$ challenges $e^{(1)}, \cdots, e^{(r-1)}$. Pick uniformly random $e_0^{(1)}, \cdots, e_0^{(r-1)}$ and compute $e_1^{(1)} := e^{(1)} \oplus e_0^{(1)}, \cdots, e_1^{(r-1)} := e^{(r-1)} \oplus e_0^{(r-1)}$. Use the simulators Sim_0, Sim_1 of Σ_0, Σ_1 to produce $(a, \{z_b^{(1)}, \cdots z_b^{(r-1)}\}) \leftarrow Sim_b(x, \{e_b^{(i)}\}_{i\in[r-1]})$ for $b = 0, 1$.*
  Output $(a, z_0^{(1)}, \cdots z_0^{(r-1)}, z_1^{(1)}, \cdots z_1^{(r-1)})$.

An example of an r strong special sound sigma protocol can be obtained from Schnorr's protocol (Example 9):

Example 19 ***[1]*** *The following sigma protocol is r special sound and $r-1$ simulatable:*

Private input to prover: *w.*

Common inputs: *Group G with generator g, large prime q, $x = g^w$, and fixed integer r.*

Commitment (by prover):

1. *Sample a random $r-1$ degree polynomial $f \in \mathbb{Z}_q[x]$ which has $f(0) = w$.*
2. *Compute commitment $\boldsymbol{a} := (g^{f(i)})_{i\in[r-1]}$.*
3. *Output $(f, \boldsymbol{a})$.*

Challenge (by verifier):

1. *Send a random $e \in \mathbb{Z}_q^*$.*

Response (by prover):

1. *Obtain e, and output $z := f(e)$.*

Verification (by verifier):

1. *Obtain $a_1, ..., a_{r-1} = \boldsymbol{a}, e$, and z.*
2. *Pick a degree $r-1$ polynomial $h \in G[x]$ which has $h(0) = x$ and $h(i) = a_i$ for each i.*

3. *Check* $h(e) = g^z$.

Completeness: *If the prover is honest, we have* $g^z = g^{f(e)}$. *Since we have* $h(j) = g^{f(j)}$ *for all* $j = 0, 1, ..., r-1$, *we must have* $h(x) = g^{f(x)}$ *since there is a unique polynomial of degree* $r-1$ *passing through* $r = (r-1)+1$ *points [27].*

r Strong Special Soundness: *Given r accepting transcripts* $(a, e, z)_{i\in[r]}$, *the r points* $(e_1, z_1), ..., (e_r, z_r)$ *uniquely determine a polynomial* $f \in \mathbb{Z}_q[x]$ *since* $z_i = f(e_i)$. *Evaluating* f *at* 0 *gives w.*

r-1 Simulatability: *The simulator takes as input* $x = g^w$ *and challenges* $e_1, ..., e_{r-1}$. *For each* $i \in [r-1]$, *it samples* $z_i \leftarrow \mathbb{Z}_q$ *and computes* g^{z_i}. *It then picks a polynomial* $h \in G[x]$ *such that* $h(0) = x = g^w$ *and* $h(e_i) = g^{z_i}$. *Finally, it computes* $\boldsymbol{a} := (h(i))_{i\in[r-1]}$ *and outputs* $\boldsymbol{a}, (z_i)_{i\in[r-1]}$. *These are identically distributed to messages from a real transcript since the real prover picks a random polynomial, so* $f(i)$ *are uniformly distributed, and hence so are* $z = f(e_i)$*'s. The simulator chooses z's uniformly.*

4.3 The New Construction: Combining Efficient Fischlin with OR Composition

This construction takes two $r+1$ strong special sound sigma protocols as input to produce a WI NI PoK with an online extractor and non-programmable random oracle. The protocol takes inspiration from the OR composition by using the simulator for the sigma protocol for which the prover does not know a witness, and uses hash (or, rather, random oracle) collisions like in the Efficient Fischlin protocol [1] to achieve online extraction. The construction is given below.

Construction 20 *Let* $\Sigma_0 = (\mathcal{P}_0, \mathcal{V}_0), \Sigma_1 = (\mathcal{P}_1, \mathcal{V}_1)$ *be two* $r+1$ *strong special sound, r simulatable, sigma protocols, with r SHVZK simulators* Sim_0, Sim_1 *respectively. Denote the new prover and verifier* P, V. *Both are provided the common input* $x = (x_0, x_1)$, *and P is provided a witness w such that either* $(x_0, w) \in Rel_0 \vee (x_1, w) \in Rel_1$. *Let* $b \in \{0,1\}$ *denote whether P has a witness for* x_0 *or for* x_1. *Let O be a random oracle of output dimension l, and let* $t := l + \lceil \log r \rceil$ *be the dimension of the challenge space, given a fixed r. Let* λ *be the security parameter, and let* l, r *be related as* $l(r-1) = \lambda$.

Prover*,* $P((x_0, x_1), w)$*:*

1. *Constructs a commitment* $\boldsymbol{a}$ *by:*
   (a) *Picking random challenges* $e^{(1)}_{1-b}, ..., e^{(r)}_{1-b} \leftarrow \{0,1\}^t$.
   (b) *Obtaining* $a_{1-b}, \{z^{(i)}_{1-b}\}_{i\in[r]} \leftarrow Sim_{1-b}(x_{1-b}, \{e^{(i)}_{1-b}\}_{i\in[r]})$
   (c) *Obtaining* $(a_b, aux) \leftarrow \mathcal{P}_b(x_b, w)$ *by running* $\mathcal{P}_b$.
   (d) *Setting* $\boldsymbol{a} = (a_0, a_1)$

2. *Define and initialise the sets* $\mathcal{E}, \mathcal{Z}_1, \cdots, \mathcal{Z}_r = \emptyset$*, and repeat until an output is produced:*

   (a) *Sample a random* $e \leftarrow \{0,1\}^t \setminus \mathcal{E}$.

   (b) *Compute* $e_b^{(1)} = e \oplus e_{1-b}^{(1)}, \cdots, e_b^{(r)} = e \oplus e_{1-b}^{(r)}$.

   (c) *Obtain* $z_b^{(1)} \leftarrow \mathcal{P}_b(aux, e_b^{(1)}), \cdots, z_b^{(r)} \leftarrow \mathcal{P}_b(aux, e_b^{(r)})$.

   (d) *For* $i = 1, \cdots, r$*, append* $(e, (e_0^{(i)}, z_0^{(i)}, e_1^{(i)}, z_1^{(i)}))$ *to* $\mathcal{Z}_i$ *and* e *to* $\mathcal{E}$.

   (e) *If there exist r pairs*

   $$(e^{(1)}, (e_0^{(1)}, z_0^{(1)}, e_1^{(1)}, z_1^{(1)})), \cdots, (e^{(r)}, (e_0^{(r)}, z_0^{(r)}, e_1^{(r)}, z_1^{(r)}))$$

   in the sets $\mathcal{Z}_1, \cdots, \mathcal{Z}_r$ *such that*

   $$O(\boldsymbol{a}, e^{(1)}, e_0^{(1)}, z_0^{(1)}, e_1^{(1)}, z_1^{(1)}) = \cdots = O(\boldsymbol{a}, e^{(r)}, e_0^{(r)}, z_0^{(r)}, e_1^{(r)}, z_1^{(r)})$$

   then set $\boldsymbol{e} = (e^{(i)})_{i \in [r]}$ *and* $\boldsymbol{z} = ((e_0^{(i)}, z_0^{(i)}, e_1^{(i)}, z_1^{(i)}))_{i \in [r]}$ *and output the proof* $\pi := (\boldsymbol{a}, \boldsymbol{e}, \boldsymbol{z})$

Verifier*,* $V((x_0, x_1), \pi)$*:*

1. *Parses* $(\boldsymbol{a}, \boldsymbol{e}, \boldsymbol{z}) = \pi$*, and* $(a_0, a_1) = \boldsymbol{a}$*,* $(e^{(i)})_{i \in [r]} = \boldsymbol{e}$*,* $((e_0^{(i)}, z_0^{(i)}, e_1^{(i)}, z_1^{(i)}))_{i \in [r]} = \boldsymbol{z}$.
2. *Checks that*

   $$O(\boldsymbol{a}, e^{(1)}, e_0^{(1)}, z_0^{(1)}, e_1^{(1)}, z_1^{(1)}) = \cdots = O(\boldsymbol{a}, e^{(r)}, e_0^{(r)}, z_0^{(r)}, e_1^{(r)}, z_1^{(r)}).$$

3. *Checks, for each i, that* $e^{(i)} = e_0^{(i)} \oplus e_1^{(i)}$ *and that* $\mathcal{V}_0(x_0, a_0, e_0^{(i)}, z_0^{(i)}) = 1$ *and* $\mathcal{V}_1(x_1, a_1, e_1^{(i)}, z_1^{(i)}) = 1$
4. *Outputs* 1 *if all above checks were satisfied, and* 0 *otherwise.*

4.4 Discussion

Instead of just one collision, a fixed number, r, of collisions are required to produce a valid proof. As will be discussed in section 4.7, this reduces the soundness error to become negligible in the security parameter λ (a measure of the difficulty an adversary will have in forging the proof). Forcing a cheating prover to guess accepting transcripts for the underlying sigma protocols r times which all have the same oracle output becomes a difficult task as r grows. Adding the commitment $\boldsymbol{a}$ to the oracle input also increases attack difficulty; provers are bound to $\boldsymbol{a}$ before they can compute challenges and responses.

The requirement for two $r+1$ strong special sound sigma protocols arises from the desire for r collisions. Assuming $r \geq 2$, an ordinary sigma protocol would not be suitable

since then there would exist an extractor capable of producing a witness upon input only 2 accepting transcripts. If $r \geq 2$ collisions are required, then at least 2 accepting transcripts are revealed by running the protocol (honestly). An adversary could then extract the witness, making WI impossible. By forcing $r+1$ strong special soundness and r simulatability, the extractor needs a minimum of $r+1$ accepting transcripts to produce a witness. Hence the adversary can no longer compute the witness using the protocol messages since only r accepting transcripts are provided.

The r simulatability of the two sigma protocols is required since the prover in Construction 20 produces r accepting transcripts and relies on r output challenge/response pairs (for the same commitment $\boldsymbol{a}$) from the simulator. It is also worth remarking that the fact that the simulator can produce r accepting transcripts without the witness means that these transcripts effectively do not yield any information about the witness at all. Hence, an extractor would not be able to extract a witness from them and this therefore forces at least $r+1$ special soundness to be satisfied by the underlying protocols.

It is also worth justifying the choice $l(r-1) = \lambda$ of security parameter. Defining in this way means extraction fails with a negligible probability in the security parameter. This will be further discussed in section 4.7. Common choices for λ are 128, 256 or 512 bits, with a higher number representing a higher security.

The following sections prove the above construction is a WI NI PoK. WI should follow from the OR Composition, and extraction from the Efficient Fischlin protocol. The proofs will not program the random oracle or make use of rewinding (of the prover). Recall this is an advantage over original protocols such as the FS transform which use a programmable random oracle.

4.5 Proof of Completeness

Completeness will follow if there always exists an r-collision of the oracle outputs. Since $\{e^{(i)}_{1-b}\}_{i\in[r]}$ are fixed in step 1a, the oracle maps a domain of size 2^t (the size of the challenge, $e^{(i)}$, space) to a codomain of size 2^l.

Since $t = l + \lceil \log_2 r \rceil$, we have that $2^t \approx r \cdot 2^l$. Hence by the pigeonhole principle, mapping a domain of size $r \cdot 2^l$ to a codomain of size 2^l results in at least one r-collision with certainty.

Section 4.8.1 gives an analysis of the expected runtime of the protocol.

4.6 Proof of Witness Indistinguishability

Theorem 21 *The above Construction 20 satisfies witness indistinguishability.*

Proof: In this proof, assume that at least one witness always exists for every x in the language for which is being proved. This witness need not be known to the prover, but it should exist. Inspiration is taken from [18].

Suppose for a contradiction that the construction is not witness indistinguishable. We will build an adversary for the SHVZK of either Σ_0 or Σ_1 to reach a contradiction.

By assumption, there exists an adversarial verifier U such that for some common input $x = (x_0, x_1)$ which has two witnesses w_A, w_B, the output from U using w_A is computationally distinguishable from the output of U using w_B.

Case 1: Assume that w_A is a witness for x_0 and that w_B is a witness for x_1.

Define the following two experiments:

- *$E_A := (P(x_0, x_1, w_A), U(x_0, x_1))$. The prover uses the witness w_A for x_0 to produce a proof. Output whatever U outputs.*
- *$E_B := (P(x_0, x_1, w_B), U(x_0, x_1))$. The prover uses the witness w_B for x_1 to produce a proof. Output whatever U outputs.*

By assumption, E_A and E_B are distinguishable.

Define an additional experiment, E^{real}, as follows. Informally, the main difference from the original protocol is that $\mathcal{P}_{1-b}$ is used instead of Sim_{1-b} to generate accepting transcripts for the case $1-b$.

***Experiment** E^{real}*

*Let w_b be the witness for x_b and w_{1-b} be the witness for x_{1-b}. Recall that w_b, w_{1-b} will each either be w_A or w_B. The new steps are indicated by an asterisk *.*

1. *Construct a commitment $\boldsymbol{a}$ by:*
   (a) ** Obtaining $(a_{1-b}, aux_{1-b}) \leftarrow \mathcal{P}_{1-b}(x_{1-b}, w_{1-b})$ by running $\mathcal{P}_{1-b}$.*
   (b) ** Picking random $e_{1-b}^{(1)}, \cdots, e_{1-b}^{(r)} \leftarrow \{0,1\}^t$.*
   (c) ** Obtaining $z_{1-b}^{(1)} \leftarrow \mathcal{P}_{1-b}(aux_{1-b}, e_{1-b}^{(1)}), \cdots, z_{1-b}^{(r)} \leftarrow \mathcal{P}_{1-b}(aux_{1-b}, e_{1-b}^{(r)})$.*
   (d) *Obtaining $(a_b, aux) \leftarrow \mathcal{P}_b(x_b, w)$ by running $\mathcal{P}_b$.*
   (e) *Setting $\boldsymbol{a} = (a_0, a_1)$.*
2. *Define and initialise the sets $\mathcal{E}, \mathcal{Z}_1, \cdots, \mathcal{Z}_r = \emptyset$, and repeat until an output is produced:*
   (a) *Sample a random $e \leftarrow \{0,1\}^t \setminus \mathcal{E}$.*
   (b) *Compute $e_b^{(1)} = e \oplus e_{1-b}^{(1)}, \cdots, e_b^{(r)} = e \oplus e_{1-b}^{(r)}$.*
   (c) *Obtain $z_b^{(1)} \leftarrow \mathcal{P}_b(aux_b^{(1)}, e_b^{(1)}), \cdots, z_b^{(r)} \leftarrow \mathcal{P}_b(aux_b^{(r)}, e_b^{(r)})$.*
   (d) *For $i = 1, \cdots, r$, append $(e, (e_0^{(i)}, z_0^{(i)}, e_1^{(i)}, z_1^{(i)}))$ to $\mathcal{Z}_i$ and e to $\mathcal{E}$.*
   (e) *If there exist r pairs*

$$(e^{(1)}, (e_0^{(1)}, z_0^{(1)}, e_1^{(1)}, z_1^{(1)})), \cdots, (e^{(r)}, (e_0^{(r)}, z_0^{(r)}, e_1^{(r)}, z_1^{(r)}))$$

in the sets $\mathcal{Z}_1, \cdots, \mathcal{Z}_r$ such that

$$O(\boldsymbol{a}, e^{(1)}, e_0^{(1)}, z_0^{(1)}, e_1^{(1)}, z_1^{(1)}) = \cdots = O(\boldsymbol{a}, e^{(r)}, e_0^{(r)}, z_0^{(r)}, e_1^{(r)}, z_1^{(r)})$$

then set $\boldsymbol{e} = (e^{(i)})_{i\in[r]}$ and $\boldsymbol{z} = ((e_0^{(i)}, z_0^{(i)}, e_1^{(i)}, z_1^{(i)}))_{i\in[r]}$ and output the proof $\pi := (\boldsymbol{a}, \boldsymbol{e}, \boldsymbol{z})$.

3. *Output whatever U outputs when provided with π.*

Since E_A and E_B are distinguishable, we must have that E^{real} is distinguishable from either E_A or E_B by transitivity, since otherwise, we would be unable to distinguish E_A from E^{real} and unable to distinguish E_B from E^{real}, and hence unable to distinguish E_A from E_B, which is false by assumption.

Assume that E^{real} and E_B are distinguishable. The other case for E_A follows similarly. We will now reach a contradiction by constructing an adversary for the SHVZK of Σ_0. A game-based definition of SHVZK is used, similar to the definition of WI. An adversary chooses and sends the common input to a challenger, who responds either with a real or simulated proof, and the adversary should guess which is the case.

Adversary A_{SHVZK}

1. *Sends x_0, w_A to the SHVZK challenger of Σ_0.*
2. *Receives back r transcripts $(a_0^{(1)}, e_0^{(1)}, z_0^{(1)}), \cdots, (a_0^{(r)}, e_0^{(r)}, z_0^{(r)})$ from the challenger. These transcripts were either generated using the real protocol Σ_0 or using the corresponding simulator Sim_0.*
3. *Constructs a commitment $\boldsymbol{a}$ by:*
   (a) *Obtaining $(a_1, aux_1) \leftarrow \mathcal{P}_1(x_1, w_B)$ by running $\mathcal{P}_1$, where aux identifies the instance of $\mathcal{P}_1$.*
   (b) *Setting $\boldsymbol{a} = (a_0, a_1)$.*
4. *Define and initialise the sets $\mathcal{E}, \mathcal{Z}_1, \cdots, \mathcal{Z}_r = \emptyset$, and repeat until an output is produced:*
   (a) *Sample a random $e \leftarrow \{0,1\}^t \setminus \mathcal{E}$.*
   (b) *Compute $e_1^{(1)} = e \oplus e_0^{(1)}, \cdots, e_1^{(r)} = e \oplus e_0^{(r)}$.*
   (c) *Obtain $z_1^{(1)} \leftarrow \mathcal{P}_1(aux_1, e_1^{(1)}), \cdots, z_1^{(r)} \leftarrow \mathcal{P}_1(aux_1, e_1^{(r)})$.*
   (d) *For $i = 1, \cdots, r$, append $(e, (e_0^{(i)}, z_0^{(i)}, e_1^{(i)}, z_1^{(i)}))$ to $\mathcal{Z}_i$ and e to $\mathcal{E}$.*
   (e) *If there exist r pairs*

$$(e^{(1)}, (e_0^{(1)}, z_0^{(1)}, e_1^{(1)}, z_1^{(1)})), \cdots, (e^{(r)}, (e_0^{(r)}, z_0^{(r)}, e_1^{(r)}, z_1^{(r)}))$$

in the sets $\mathcal{Z}_1, \cdots, \mathcal{Z}_r$ such that

$$O(\boldsymbol{a}, e^{(1)}, e_0^{(1)}, z_0^{(1)}, e_1^{(1)}, z_1^{(1)}) = \cdots = O(\boldsymbol{a}, e^{(r)}, e_0^{(r)}, z_0^{(r)}, e_1^{(r)}, z_1^{(r)})$$

then set $\boldsymbol{e} = (e^{(i)})_{i\in[r]}$ and $\boldsymbol{z} = ((e_0^{(i)}, z_0^{(i)}, e_1^{(i)}, z_1^{(i)}))_{i\in[r]}$ and output the proof $\pi := (\boldsymbol{a}, \boldsymbol{e}, \boldsymbol{z})$.

5. *Send the proof π to U, and output whatever U outputs.*

The adversary A_{SHVZK} will receive from the SHVZK challenger either a transcript from the real sigma protocol Σ_0 or from the corresponding simulator Sim_0 for x_0 and uses the real sigma protocol Σ_1 always for x_1.

First, assume that the SHVZK challenger returns transcripts produced using the real protocol Σ_0. Then, the output produced from the adversary A_{SHVZK} is indistinguishable from the output produced in the experiment E^{real}. This is because the real prover was used in E^{real} for both instances x_0 and x_1. If a real proof is returned to the adversary, then it too computes transcripts for two real proofs.

Next, assume that the SHVZK challenger returns a transcript produced using the simulator Sim_0. Then, the output produced by the adversary is indistinguishable from the output produced in the experiment E_B. This is because the simulator was used for case x_0 in E_B and the real protocol was used for case x_1, since w_B was a witness for x_1.

Since we assumed E^{real} and E_B were distinguishable, we are therefore able to distinguish between when the challenger produces a real or simulated transcript and hence have broken SHVZK.

***Case 2: Assume that** w_A **and** w_B **are both witnesses for** x_0. Additionally assume that there exists at least one witness w_C for x_1. The other case where w_A and w_B are both witness for x_1 follows similarly.*

The proof is now very similar to the first case in logic, but with minor details to which prover or simulator is being used.

Define the following two experiments:

- *$E_A := (P(x_0, x_1, w_A), U(x_0, x_1))$. The prover uses the witness w_A for x_0 to produce a proof. Output whatever U outputs.*
- *$E_B := (P(x_0, x_1, w_B), U(x_0, x_1))$. The prover uses the witness w_B for x_0 to produce a proof. Output whatever U outputs.*

By assumption, E_A and E_B are distinguishable.

***Experiment** $E^{real'}$*

*The new and changed steps are indicated by an asterisk *.*

1. *Construct a commitment $\boldsymbol{a}$ by:*

   (a) * *Obtaining $(a_0, aux_0) \leftarrow \mathcal{P}_0(x_0, w_B)$ by running $\mathcal{P}_0$.*

(b) * Picking random $e_0^{(1)}, \cdots, e_0^{(r)} \leftarrow \{0,1\}^t$.

(c) * Obtaining $z_0^{(1)} \leftarrow \mathcal{P}_0(aux_0, e_0^{(1)}), \cdots, z_0^{(r)} \leftarrow \mathcal{P}_0(aux_0, e_0^{(r)})$.

(d) * Obtaining $(a_1, aux_1) \leftarrow \mathcal{P}_1(x_1, w_C)$ by running $\mathcal{P}_1$.

(e) Setting $\boldsymbol{a} = (a_0, a_1)$.

2. Define and initialise the sets $\mathcal{E}, \mathcal{Z}_1, \cdots, \mathcal{Z}_r = \emptyset$, and repeat until an output is produced:

(a) Sample a random $e \leftarrow \{0,1\}^t \setminus \mathcal{E}$.

(b) * Compute $e_1^{(1)} = e \oplus e_0^{(1)}, \cdots, e_1^{(r)} = e \oplus e_0^{(r)}$.

(c) * Obtain $z_1^{(1)} \leftarrow \mathcal{P}_1(aux_1^{(1)}, e_1^{(1)}), \cdots, z_1^{(r)} \leftarrow \mathcal{P}_1(aux_1^{(r)}, e_1^{(r)})$.

(d) For $i = 1, \cdots, r$, append $(e, (e_0^{(i)}, z_0^{(i)}, e_1^{(i)}, z_1^{(i)}))$ to $\mathcal{Z}_i$ and e to $\mathcal{E}$.

(e) If there exist r pairs

$$(e^{(1)}, (e_0^{(1)}, z_0^{(1)}, e_1^{(1)}, z_1^{(1)})), \cdots, (e^{(r)}, (e_0^{(r)}, z_0^{(r)}, e_1^{(r)}, z_1^{(r)}))$$

in the sets $\mathcal{Z}_1, \cdots, \mathcal{Z}_r$ such that

$$O(\boldsymbol{a}, e^{(1)}, e_0^{(1)}, z_0^{(1)}, e_1^{(1)}, z_1^{(1)}) = \cdots = O(\boldsymbol{a}, e^{(r)}, e_0^{(r)}, z_0^{(r)}, e_1^{(r)}, z_1^{(r)})$$

then set $\boldsymbol{e} = (e^{(i)})_{i \in [r]}$ and $\boldsymbol{z} = ((e_0^{(i)}, z_0^{(i)}, e_1^{(i)}, z_1^{(i)}))_{i \in [r]}$ and output the proof $\pi := (\boldsymbol{a}, \boldsymbol{e}, \boldsymbol{z})$.

3. Output whatever U outputs when provided with π.

Since E_A and E_B are distinguishable, we must have that $E^{real'}$ is distinguishable from either E_A or E_B. Assume that $E^{real'}$ and E_B are distinguishable. The other case for E_A follows similarly. We will now reach a contradiction by constructing an adversary for the SHVZK of Σ_0.

Adversary A'_{SHVZK}

1. Sends x_0, w_B to the SHVZK challenger of Σ_0.

2. Receives back r transcripts $(a_0^{(1)}, e_0^{(1)}, z_0^{(1)}), \cdots, (a_0^{(r)}, e_0^{(r)}, z_0^{(r)})$ from the challenger. These transcripts were either generated using the real protocol Σ_0 or using the corresponding simulator Sim_0.

3. Constructs a commitment $\boldsymbol{a}$ by:

(a) Obtaining $(a_1, aux_1) \leftarrow \mathcal{P}_1(x_1, w_C)$ by running $\mathcal{P}_1$.

(b) Setting $\boldsymbol{a} = (a_0, a_1)$.

4. Define and initialise the sets $\mathcal{E}, \mathcal{Z}_1, \cdots, \mathcal{Z}_r = \emptyset$, and repeat until an output is

produced:

(a) Sample a random $e \leftarrow \{0,1\}^t \setminus \mathcal{E}$.

(b) Compute $e_1^{(1)} = e \oplus e_0^{(1)}, \cdots, e_1^{(r)} = e \oplus e_0^{(r)}$.

(c) Obtain $z_1^{(1)} \leftarrow \mathcal{P}_1(aux_1, e_1^{(1)}), \cdots, z_1^{(r)} \leftarrow \mathcal{P}_1(aux_1, e_1^{(r)})$.

(d) For $i = 1, \cdots, r$, *append* $(e, (e_0^{(i)}, z_0^{(i)}, e_1^{(i)}, z_1^{(i)}))$ *to* $\mathcal{Z}_i$ *and* e *to* $\mathcal{E}$.

(e) If there exist r pairs

$$(e^{(1)}, (e_0^{(1)}, z_0^{(1)}, e_1^{(1)}, z_1^{(1)})), \cdots, (e^{(r)}, (e_0^{(r)}, z_0^{(r)}, e_1^{(r)}, z_1^{(r)}))$$

in the sets $\mathcal{Z}_1, \cdots, \mathcal{Z}_r$ *such that*

$$O(\boldsymbol{a}, e^{(1)}, e_0^{(1)}, z_0^{(1)}, e_1^{(1)}, z_1^{(1)}) = \cdots = O(\boldsymbol{a}, e^{(r)}, e_0^{(r)}, z_0^{(r)}, e_1^{(r)}, z_1^{(r)})$$

then set $\boldsymbol{e} = (e^{(i)})_{i \in [r]}$ *and* $\boldsymbol{z} = ((e_0^{(i)}, z_0^{(i)}, e_1^{(i)}, z_1^{(i)}))_{i \in [r]}$ *and output the proof* $\pi := (\boldsymbol{a}, \boldsymbol{e}, \boldsymbol{z})$.

5. Send the proof π *to* U, *and output whatever* U *outputs.*

When the SHVZK challenger returns transcripts produced using the real protocol of Σ_0, *then the output produced by the adversary* A'_{SHVZK} *is indistinguishable from the output produced in the experiment* $E^{real'}$. *This is because the real prover was used in* $E^{real'}$ *for both cases* x_0 *and* x_1.

When the challenger returns a transcript produced using the simulator Sim_0, *the output produced by the adversary is indistinguishable from the output produced in the experiment* E_B. *This is because the simulator was used for case* x_0 *in* E_B *and the real protocol was used for case* x_1.

Since we assumed $E^{real'}$ *and* E_B *are distinguishable, we are therefore able to distinguish between when the challenger produces a real or simulated transcript, and hence have broken SHVZK.*

4.7 Proof of Soundness / Extractor

Soundness will follow if there exists an extractor that, when provided an accepting transcript for common input x, and access to the queries made by the prover to the random oracle in producing the transcript, will output a valid witness w for x. The extractor is given below and is then shown to fail with negligible probability.

The extractor calls upon the extractors of Σ_0 and Σ_1. Successful extractors exist for Σ_0, Σ_1 since these are two $r+1$ strong special sound, r simulatable, sigma protocols. The only issue is that these extractors require $r+1$ accepting transcripts and only r transcripts are provided through a protocol execution. This is resolved by looking at the queries the prover makes.

Extractor The extractor receives as input $x = (x_0, x_1)$ and a proof $\pi = (\boldsymbol{a}, \boldsymbol{e}, \boldsymbol{z})$, and access to queries Q to the random oracle O made by the prover.

The extractor:

1. Parses $(a_0, a_1) = \boldsymbol{a}$, $(e^{(i)})_{i\in[r]} = \boldsymbol{e}$, $((e_0^{(i)}, z_0^{(i)}, e_1^{(i)}, z_1^{(i)}))_{i\in[r]} = \boldsymbol{z}$.
2. Searches Q for a query $O(\boldsymbol{a}, e, e_0, z_0, e_1, z_1)$ such that the two transcripts a_0, e_0, z_0 and a_1, e_1, z_1 are accepting according to $\mathcal{V}_0$, respectively $\mathcal{V}_1$, and such that the query is not found in π;
   i.e. such that $e_0 \notin \{e_0^{(i)}\}_{i\in[r]} \vee e_1 \notin \{e_1^{(i)}\}_{i\in[r]} \vee z_0 \notin \{z_0^{(i)}\}_{i\in[r]} \vee z_1 \notin \{z_1^{(i)}\}_{i\in[r]}$.
3. Uses the extractor of Σ_0 on input $x_0, (a_0, e_0, z_0), (a_0^{(i)}, e_0^{(i)}, z_0^{(i)})_{i\in[r]}$ or the extractor of Σ_1 on input $x_1, (a_1, e_1, z_1), (a_1^{(i)}, e_1^{(i)}, z_1^{(i)})_{i\in[r]}$ to output a valid witness.

It remains to analyse when the extractor fails. This can only happen when it fails to find a matching query in step 2. This in turn can only happen when the prover P finds r hash collisions by making only r queries to the random oracle O, so there is no $r+1$-th query for the extractor to use.

Given the random oracle acts like a random function, the output of each query is an independently chosen l-bit string. The probability that r such strings are equal is $(\frac{1}{2^l})^{r-1} = 2^{-\lambda}$, which is negligible in the security parameter. Hence the extraction error is negligible.

The extractor is online and does not make any rewinds of the prover as it executes. The random oracle is also not programmed in either the proof of WI or Extractability. Hence, we have shown that the construction is a WI NI PoK with online extractor and non-programmable random oracle as desired.

4.8 Runtime Analysis

4.8.1 Empirical Results

Assuming the simulators Sim_0, Sim_1 and provers $\mathcal{P}_0$, $\mathcal{P}_1$ run efficiently, the computationally intensive part of the protocol is the loop finding collisions in the random oracle output.

In order to analyse how long this might take, we can start by appealing to the so-called birthday theorem [16], which gets its name from bounding the probability two people in a room have the same birthday.

Theorem 22 ***[16] Birthday Theorem*** *If q elements $y_1, ..., y_q$ are chosen uniformly and independently randomly from a set of size N, then the probability that there are $i \neq j$ such that $y_i = y_j$ is* ***at most*** $\frac{q^2}{2N}$. *If $q \leq \sqrt{2N}$ then the probability is* ***at least*** $\frac{q(q-1)}{4N}$.

The birthday theorem can be used to analyse how many elements need to be sampled before a collision becomes likely. It essentially says the probability of a collision grows as $O(q^2)$. Figure 4.1 plots the probability of obtaining two collisions when sampling q

items from a set of size N, according to the birthday theorem. Notice that we require far fewer than N samples to produce a collision with non-negligible probability.

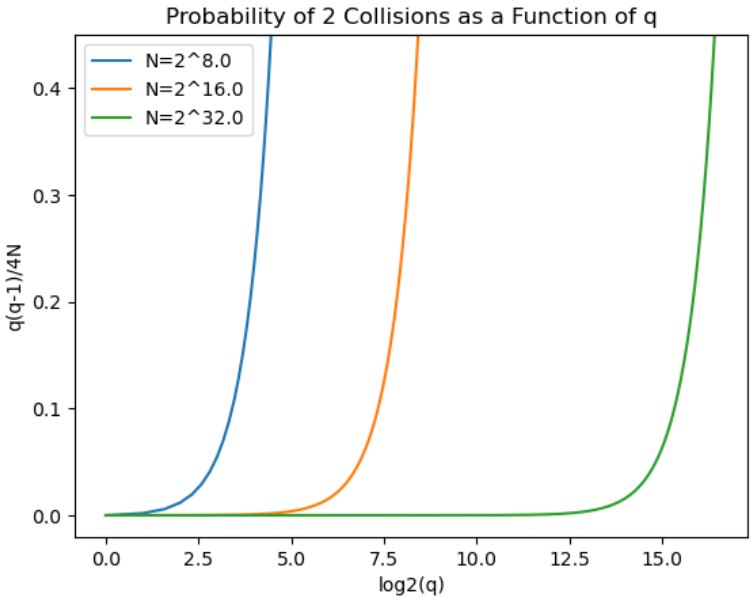

Figure 4.1: Visualisation of the birthday theorem, plotting the lower-bound probability as a function of q for various N.

The output of the random oracle in Construction 20 is completely independent given distinct inputs. It therefore remains to analyse the likelihood of obtaining r identical strings when picking independently and uniformly from a set of size 2^l. Note that this is exactly the setup of the birthday theorem if $r = 2$, and we have $N = 2^l$ possible oracle outputs.

For a generic r, the probability of obtaining an r-collision is estimated as follows. After making q queries to the random oracle and obtaining the results, the number of unordered r-tuples of results (i.e. number of ways to group r results) is given by

$$\binom{q}{r} = \frac{q(q-1)(q-2)\cdots(q-r+1)}{r!} \approx \frac{q^r}{r!}.$$

The probability that any given tuple is an r-collision is $(\frac{1}{2^l})^{r-1} = 2^{-l(r-1)} = 2^{-\lambda}$. Hence the probability of obtaining an r-collision after making q queries is approximately

$$\frac{q^r}{r!}2^{-l(r-1)} = \frac{q^r}{r!}2^{-\lambda}. \tag{4.1}$$

For a probability of $\frac{1}{2}$, we can rearrange to find $q = (r! \cdot 2^{l(r-1)-1})^{\frac{1}{r}}$.

Figure 4.2 gives an empirical comparison of the mean number of queries by Fischlin's prover and the prover in Construction 20 for a fixed length $l = 10$ as a function of r. The expected number of queries for each protocol is also plotted - according to $r \cdot 2^l$ for Fischlin, and equation 4.1 rearranged for a probability of $\frac{1}{2}$ for Construction 20.

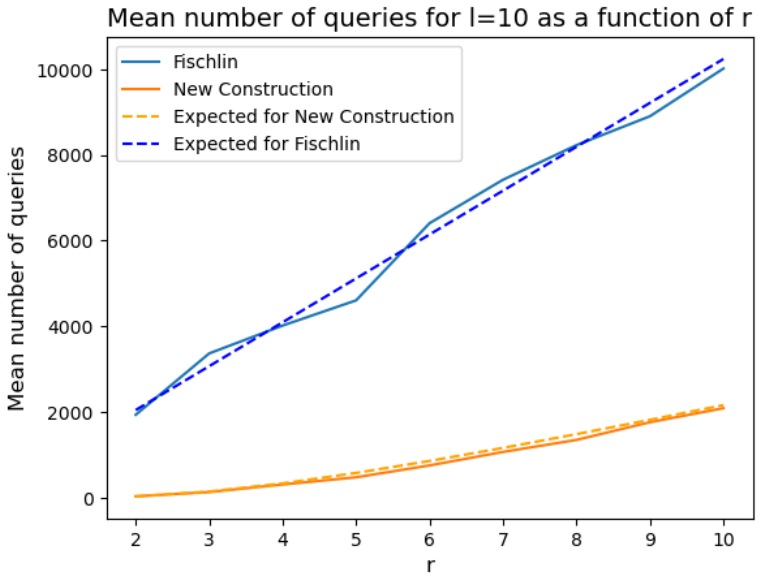

Figure 4.2: Comparison of the mean number of queries made over 100 trials.

Construction 20 requires significantly fewer queries to produce a collision than Fischlin's protocol requires to find a preimage. The reason for this is that there is a polynomial speed-up in finding collisions compared to preimages. A preimage attack requires searching 2^l queries in expectation for an output length l (and for Fischlin this rises to $r \cdot 2^l$ since r repetitions are required [1]), whereas for a collision the query complexity is $2^{\frac{l}{2}}$ in expectation.

Table 4.1 gives a numerical comparison of the mean number of queries taken to find a collision across 10 independent trials, for different (l, r) pairs, for implementations of both Fischlin's construction and the new construction. The values in the table agree to the same order of magnitude with those from the birthday equation graph.

The code for this section is given in Appendix A.

4.8.2 Polynomial Runtime

It is also possible to ensure the proof runs efficiently, i.e. polynomially in λ, under certain assumptions:

Theorem 23 *Construction 20 runs in time $O(poly(\lambda))$, polynomial in the security parameter, assuming $l = O(\log(\lambda))$.*

Proof: Rearranging equation 4.1 for a probability of 1 we find $q = (r! \cdot 2^{l(r-1)})^{\frac{1}{r}} = (r! \cdot 2^{\lambda})^{\frac{1}{r}}$. Since $\lambda = l(r-1)$, this can be approximated by $(r! \cdot 2^{\lambda})^{\frac{1}{r}} \approx (r!)^{\frac{1}{r}} \cdot 2^l$ under the approximation $r \approx r - 1$.

***Claim**: $(r!)^{\frac{1}{r}} \cdot 2^l \leq r \cdot 2^l$ for all integers $r \geq 1$.*

Proof of claim: Clearly $r! = r(r-1)(r-2)\cdots 2 \cdot 1 \leq r^r$. Hence $(r!)^{\frac{1}{r}} \leq (r^r)^{\frac{1}{r}} = r$, and $(r!)^{\frac{1}{r}} \cdot 2^l \leq r \cdot 2^l$.

l	r	Fischlin # Queries	New Construction # Queries
4	2	27	5
4	4	57	19
4	8	124	62
4	10	165	94
8	2	613	22
8	4	800	128
8	8	2,075	456
8	10	2,507	706
16	2	111,253	232
16	4	316,184	5,816
16	8	507,507	44,562
16	10	645,761	78,064

Table 4.1: Comparison of number of queries to the random oracle for various l, r pairs between Fischlin's construction and the new construction. Query counts are the mean of 10 independent trials.

Now under the assumption that $l = O(\log(\lambda))$, we find the number of required queries to be in the order of at most $r \cdot 2^l = O(r \cdot 2^{\log\lambda}) = O(r \cdot \lambda)$ and approximating $r \approx \frac{\lambda}{l}$ gives $r \cdot \lambda \approx \frac{\lambda}{\log\lambda} \cdot \lambda = \frac{1}{\log\lambda}\lambda^2$. Now, $\frac{1}{\log\lambda} \leq 1$ for $\lambda \geq 2$ (which will be the case in all applications), hence we achieve the polynomial query complexity $O(\lambda^2)$ for $\lambda \geq 2$.

4.9 Limitations

While the protocol achieves witness indistinguishability without programming the random oracle, it is worth recognising certain limitations.

- Due to the proof-of-work nature of the protocol, it will not be well-suited to time-critical scenarios. This is especially true as the length parameter l grows. However, there are still possible applications in key distribution protocols or signature schemes for example, where the protocol would not need to be run repeatedly in a short time frame.

  The Fiat Shamir transform is much more efficient, with only a single hash query, however, as previously discussed, it does not enjoy non-programmability and online extraction.

- The construction does not satisfy ZK, but instead is WI. While it would be useful to have ZK, we must remember that ZK is impossible in a NI setting for non-trivial languages, without setup assumptions. While the random oracle model is assumed, it is still impossible to satisfy ZK with a non-programmable random oracle.

- The proof of witness indistinguishability assumes that a witness exists for every x in the language. For an arbitrary language it is technically possible that a witness does not exist, (for example c has no witness for the language $\{a, b, c\}$ with

relation $\{(a,x),(b,y)\}$), however, for most use cases this will not be an issue. Every formula in the language *SAT* consisting of satisfiable Boolean statements has a witness, namely its assignment, and *SAT* is NP-complete. Hence, the proof is at least applicable to all *NP* languages.

Alternatively, perhaps we could use the same x' for both inputs x_0 and x_1, (i.e. set $x = (x', x')$ in the protocol) and pick an x' which has multiple witnesses.

- Due to limited memory capacity, computing average query counts in the implementation of the protocols for larger parameters is difficult. The length parameter l is set relatively small to enable comparisons on a low-powered device. However, notice that for the parameter values used in this report, the implementations follow the expected query counts closely, so the expected counts could potentially be used as a proxy estimate. It would also be useful to compute the mean counts using more trials. Additionally, it is worth considering that the randomness used in the implementations is not *true-randomness*, but pseudorandomness.
- Improvements could be made in the implementation to optimise query complexity. There could be more optimal methods of checking if an r collision exists. In the current implementation, every new transcript for the new construction is queried, and counts are checked using a lookup table.

Chapter 5

Conclusion

5.1 Revisiting the Project Aims

The goal of the project was to combine the efficiency gains made by the Efficient Fischlin protocol [1] over Fischlin's original protocol [3] with the WI properties of the OR Composition [9].

Construction 20 successfully achieves WI and soundness in the non-programmable random oracle model, making use of an online extractor without rewinding which fails only with negligible probability in the security parameter. Empirical analysis also shows that Construction 20 outperforms Fischlin's protocol in terms of prover query complexity as expected.

5.2 Future Work

An assumption of the construction in this report is an $r+1$ strong special sound sigma protocol. It would be interesting to investigate if it is possible to transform an arbitrary sigma protocol into an $r+1$ strong special sound one. As showed in section 4.2, a sigma protocol with deterministic responses satisfies the r strong special soundness property (although this says nothing about r simulatability). Work in this direction could be fruitful.

Other ideas could include building a system of equations which has a solution only when r equations are provided or perhaps an (insecure) encryption/hiding scheme which hides messages when provided up to $r-1$ ciphertexts, but which reveals the messages upon the r-th ciphertext example.

Example 19 is an r strong special sound sigma protocol based on Schnorr's protocol for the Discrete Logarithm (DLog) problem. Hence, in the absence of a universal transform for an arbitrary sigma protocol, it would suffice to encode the desired relation into the DLog relation, then apply the r strong special sound sigma protocol.

It would also be useful to test the oracle query complexity for larger parameters to give further confidence in the efficiency of the new construction.

Bibliography

[1] A. Shelat Y. Kondi. Improved straight-line extraction in the random oracle model with applications to signature aggregation. *ASIACRYPT*, 2022.

[2] M. Ciampi. Efficient nizk arguments with straight-line simulation and extraction. *CANS*, 2022.

[3] M. Fischlin. Communication-efficient non-interactive proofs of knowledge. *Crypto*, 2005.

[4] O. Goldreich. Proofs that yield nothing but their validity or all languages in np have zero-knowledge proof systems. *ACM Volume 38*, 1991.

[5] B. Roy S. Panja. A secure end-to-end verifiable e-voting system using zero knowledge based blockchain. *Journal of Information Security and Applications*, 2021.

[6] O. Goldreich. How to play any mental game. *STOC*, 1987.

[7] C. Rackoff S. Goldwasser, S. Micali. The knowledge complexity of interactive proof systems. *SIAM Journel pp186-208*, 1989.

[8] R. Cramer. *Modular Design of Secure yet Practical Cryptographic Protocols*. PhD thesis, The University of Amsterdam, 1996.

[9] I. Damgard. On sigma protocols. *CPT*, 2010.

[10] A. Shamir A. Fiat. How to prove yourself: Practical solutions to identification and signature problems. *Crypto*, 1986.

[11] Y. Lindell. How to simulate it – a tutorial on the simulation proof technique. *Tutorials on the Foundations of Cryptography*, 2017.

[12] O. Goldreich. *Foundations of Cryptography*. Cambridge University Press, 2009.

[13] M. Ibrahem. Modification of diffie-hellman key exchange for zkp. *International Conference on Future Communication Networks*, 2012.

[14] D. Kogan. Lecture 5: Proofs of knowledge, schnorr's protocol, nizk. *CS 355: Topics in Cryptography*, 2019.

[15] C. Schnorr. Efficient identification and signatures for smart cards. *Crypto 1989*.

[16] Y. Lindell J. Katz. *Introduction to Modern Cryptography*. Chapman Hall/CRC, 2015.

[17] U. Feige. Witness indistinguishable and witness hiding protocols. *STOC*, 1990.

[18] J. Garay. Strengthening zero-knowledge protocols using signatures. 2005.

[19] O. Goldreich. On the composition of zero-knowledge proof systems. *Proc. of the 17th ICALP, Lecture Notes in Computer Science*, 1990.

[20] J. Katz. Lecture 21, advanced topics in cryptography, cmsc 858k. 2004.

[21] M. Bellare P. Rogaway. Random oracles are practical: A paradigm for designing efficient protocols. *1st Conf. - Comm. Security*, 1993.

[22] D. Pointcheval. Security proofs for signature schemes. *Eurocrypt*, 1996.

[23] T. Attema et. al. The fiat–shamir transformation of special-sound interactive proofs. *Journal of Cryptography*, 2024.

[24] Y. Lindell. An efficient transform from sigma protocols to nizk with a crs and non-programmable random oracle. 2015.

[25] S. Goldwasser and Y. Kalai. On the (in)security of the fiat-shamir paradigm. *4th FOCS*, 2004.

[26] M. Blum. Non interactive zero knowledge and its applications. *STOC*, 1988.

[27] D. Tse S. Rao. Discrete mathematics and probability theory lecture notes 6. *Stanford CS 70*, 2009.

Appendix A

Implementation Code

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from Crypto.Util.number import getPrime
4
5 class String():
6     def __init__(self, length):
7         self.length = length
8         self.string = ""
9     def set(self, string):
10        self.string = string
11    def increment(self):
12        self.string = bin(int(self.string, 2) + 1)[2:].zfill(self.
      length)
13    def add(self, other_string):
14        self.string = bin(int(self.string, 2) + int(other_string, 2)
      )[2:].zfill(self.length)
15    def mult(self,other_string):
16        self.string = bin(int(self.string, 2) * int(other_string, 2)
      )[2:].zfill(self.length)
17    def xor(self, other_string):
18        self.string = bin(int(self.string, 2) ^ int(other_string, 2)
      )[2:].zfill(self.length)
19    def power(self, power):
20        self.string = bin(int(self.string, 2) ** power)[2:].zfill(
      self.length)
21    def get(self):
22        return self.string
23    def set_random(self):
24        bits = np.random.randint(2, size=self.length)
25        self.string = ''.join(map(str, bits))
26    def set_from_int(self, integer):
27        self.string = bin(integer)[2:].zfill(self.length)
28
29 class RandomOracle():
30    def __init__(self, length):
31        self.length = length
32        self.lookup = {}
33    def get(self, string):
34        if string.get() in self.lookup:
```

```
35              return self.lookup[string.get()]
36          else:
37              s = String(self.length)
38              s.set_random()
39              self.lookup[string.get()] = s.get()
40              return s.get()
41
42  class SigmaProtocol():
43      """Schnorr's Sigma Protocol for proving knowledge of a witness w
        such that g^w = x (mod q) where q is a prime number.
44      @input x : String, (mod q)
45      @input w : String, (mod q)
46      @input q : String, prime number
47      @input generator : String, generator of Z_q
48      @input random_oracle : RandomOracle, a random oracle
49      @input q_len : int, bit length of q
50      """
51      def __init__(self, q_len=None,random_oracle=None,x=None,w=None,
        prime=None,generator=None):
52
53          # Allow auto-generation of q,x,w,g if not provided
54          if (x is None) or (w is None) or (prime is None) or(
        generator is None):
55              self.q_len = q_len
56              self.q = getPrime(q_len)
57              self.random_oracle = random_oracle
58              self.random_strings = {}
59
60              # Witness w in Z_q such that g^w = x (where ^ is
        repeated addition mod q)
61              self.witness = String(q_len)
62              self.witness.set_random()
63              self.witness.set_from_int(int(self.witness.get(),2) %
        self.q)
64
65              # Generator of Z_q (under addition mod q) - any integer
        coprime to q will do
66              self.generator = String(q_len)
67              self.generator.set_random()
68              self.generator.set_from_int(int(self.generator.get(),2)
        % self.q)
69
70              # Common input = g^w, and the group operation is
        addition mod q
71              self.x = String(q_len)
72              self.x.set(self.generator.get())
73              self.x.mult(self.witness.get())
74              self.x.set_from_int(int(self.x.get(),2) % self.q)
75          else:
76              self.x=x
77              self.witness=w
78              self.q=prime
79              self.q_len  = q_len
80              self.generator = generator
81              self.random_oracle = random_oracle
82              self.random_strings = {}
```

```
83
84
85      def get_commitment(self):
86          r = String(self.q_len)
87          r.set_random()
88          r.set_from_int(int(r.get(),2) % self.q)
89          commitment = String(self.q_len)
90          commitment.set(self.generator.get())
91          commitment.mult(r.get())
92          commitment.set_from_int(int(commitment.get(),2) % self.q)
93          self.random_strings[commitment.get()] = r.get()
94          return commitment.get()
95
96      def get_challenge(self, commitment):
97          challenge = String(self.q_len)
98          challenge.set_random()
99          return challenge.get()
100
101     def get_response(self, commitment, challenge):
102         response = String(self.q_len)
103         response.set(self.witness.get())
104         response.mult(challenge)
105         response.add(self.random_strings[commitment])
106         response.set_from_int(int(response.get(),2) % self.q)
107         return response.get()
108
109     # Verifier.  Note that x should also be a parameter,
110     # but this is instead hardcoded into the Class.
111     def verify(self, commitment, challenge, response):
112         lhs = String(self.q_len)
113         lhs.set(self.generator.get())
114         lhs.mult(response)
115         lhs.set_from_int(int(lhs.get(),2) % self.q)
116         rhs = String(self.q_len)
117         rhs.set(self.x.get())
118         rhs.mult(challenge)
119         rhs.add(commitment)
120         rhs.set_from_int(int(rhs.get(),2) % self.q)
121         return lhs.get() == rhs.get()
122
123 class Fischlin():
124     def __init__(self,oracle_length=16,repetitions_required=5,
    timeout_limit=1e10):
125         self.oracle_length = oracle_length
126         self.repetitions_required = repetitions_required
127         self.timeout_limit = timeout_limit
128         self.random_oracle = RandomOracle(oracle_length)
129         self.sigma = SigmaProtocol(oracle_length,self.random_oracle)
130
131     def run(self):
132         # store the number of queries required to find a hash
    preimage for each of the repetitions
133         queriesOnEachRepetition = [0 for i in range(self.
    repetitions_required)]
134
135         # --- protocol ---
```

```
136         commitments = [self.sigma.get_commitment() for i in range(
    self.repetitions_required)]
137         fcommitments = commitments
138         fchallenges = [-1 for i in range(self.repetitions_required)]
139         fresponses = [-1 for i in range(self.repetitions_required)]
140         for i in range(self.repetitions_required):
141             found = False
142             while not found:
143                 if fchallenges[i] >= self.timeout_limit:
144                     print("timeout",i)
145                     break
146                 fchallenges[i] = fchallenges[i]+1
147 
148                 # get binary string
149                 challenge_i_string_obj = String(self.oracle_length)
150                 challenge_i_string_obj.set_from_int(fchallenges[i])
151                 challenge_i_string = challenge_i_string_obj.get()
152 
153                 # get response
154                 response_i = self.sigma.get_response(commitments[i],
    challenge_i_string)
155 
156                 # check oracle
157                 raw_oracle_query = "".join(list(map(str, ["".join(
    fcommitments),i,challenge_i_string,response_i])))
158                 oracle_query = String(len(raw_oracle_query))
159                 oracle_query.set(raw_oracle_query)
160                 oracle_output = self.random_oracle.get(oracle_query)
161                 queriesOnEachRepetition[i] += 1
162 
163                 if oracle_output == "0"*self.oracle_length:
164                     fchallenges[i] = challenge_i_string
165                     fresponses[i] = response_i
166                     found = True
167 
168         meanNrQueries = np.mean(queriesOnEachRepetition)
169         totalNrQueries = sum(queriesOnEachRepetition)
170         return fcommitments,fchallenges,fresponses, meanNrQueries,
    totalNrQueries
171 
172 
173 class NewConstruction():
174     def __init__(self,oracle_length=16,repetitions_required=5,
    timeout_limit=1e10):
175         self.oracle_length = oracle_length
176         self.repetitions_required = repetitions_required
177         self.timeout_limit = timeout_limit
178         self.random_oracle = RandomOracle(oracle_length)
179 
180         # set up sigma protocols
181         q_len = oracle_length
182         q = getPrime(q_len)
183         generator = String(q_len)
184         generator.set_random()
185         generator.set_from_int(int(generator.get(),2) % q)
186         witness0 = String(q_len)
```

```
187         witness0.set_random()
188         witness0.set_from_int(int(witness0.get(),2) % q)
189         witness1 = String(q_len)
190         witness1.set_random()
191         witness1.set_from_int(int(witness1.get(),2) % q)
192         x0 = String(q_len)
193         x0.set(generator.get())
194         x0.mult(witness0.get())
195         x0.set_from_int(int(x0.get(),2) % q)
196         x1 = String(q_len)
197         x1.set(generator.get())
198         x1.mult(witness1.get())
199         x1.set_from_int(int(x1.get(),2) % q)
200
201
202         self.sigma0 = SigmaProtocol(q_len,self.random_oracle,x0,
    witness0,q,generator)
203         self.sigma1 = SigmaProtocol(q_len,self.random_oracle,x1,
    witness1,q,generator)
204
205
206     """
207     Note that the schnorr protocol (for which the sigma protocols
    are based) is not a
208     r+1 strong special sound sigma protocol (because not r
    simulatable), so here, just random
209     strings are returned to allow analysis.  Future verification
    would then fail.
210     """
211     def simulator(self,rchallenges):
212         a = String(self.oracle_length)
213         a.set_random()
214         a.set_from_int(int(a.get(),2) % self.sigma0.q)
215
216         # generate len(rchallenges) random strings
217         zs = []
218         for i in range(len(rchallenges)):
219             z = String(self.oracle_length)
220             z.set_random()
221             z.set_from_int(int(z.get(),2) % self.sigma0.q)
222             zs.append(z.get())
223         return a.get(),zs
224
225     def run(self):
226         # assume b=0
227         queryCount = 0
228         try_count = 0
229         output_counts = {}
230
231         # 1.a Generate random commitments for (1-b=1)
232         challenges1 = []
233         for i in range(self.repetitions_required):
234             c = String(self.oracle_length)
235             c.set_random()
236             c.set_from_int(int(c.get(),2) % self.sigma1.q)
237             challenges1.append(c.get())
```

```
238
239         # 1.b Generate commitment and responses using the simulator
240         commitment1, responses1 = self.simulator(challenges1)
241
242         # 1.c Generate commitment for (b=0)
243         commitment0 = self.sigma0.get_commitment()
244
245         # 1.d set commitment
246         nccommitment = [commitment0,commitment1]
247
248         # 2 define empty sets
249         E = []
250         Zs = [[] for i in range(self.repetitions_required)]
251
252         # 2.a generate a new challenge
253         time = 0
254         while time < self.timeout_limit:
255             time += 1
256             try_count += 1
257             ncchallenge = String(self.oracle_length)
258             ncchallenge.set_random()
259             ncchallenge.set_from_int(int(ncchallenge.get(),2) % self
    .sigma0.q)
260
261             # 2.b compute challenges for (b=0)
262             challenges0 = []
263             for i in range(self.repetitions_required):
264                 c = String(self.oracle_length)
265                 c.set(ncchallenge.get())
266                 c.xor(challenges1[i])
267                 challenges0.append(c.get())
268
269             # 2.c compute responses for (b=0)
270             responses0 = []
271             for i in range(self.repetitions_required):
272                 r = self.sigma0.get_response(commitment0,challenges0
    [i])
273                 responses0.append(r)
274
275             # 2.d append to sets
276             for i in range(self.repetitions_required):
277                 E.append(ncchallenge.get())
278                 Zs.append([ncchallenge.get(),challenges0[i],
    responses0[i],challenges1[i],responses1[i]])
279
280             # 2.e check if there exists r tuples in Zs such that the
     oracle output is 0
281             # hash the new Zs entries
282             for z in Zs[-self.repetitions_required:]:
283                 raw_query = "".join([*nccommitment,*z])
284                 query = String(len(raw_query))
285                 query.set(raw_query)
286                 result = self.random_oracle.get(query)
287                 queryCount += 1
288                 if result in output_counts:
289                     output_counts[result] += 1
```

```
290                 else:
291                     output_counts[result] = 1
292
293             # check if any of the counts are equal to
    repetitions_required
294             for key in output_counts:
295                 if output_counts[key] == self.repetitions_required:
296                     return queryCount, try_count
297
298 rs = [2,3,4,5,6,7,8,9,10]
299 l = 10
300
301 def expected_number_of_queries(l,r):
302     return (2**(l*(r-1)-1) * np.math.factorial(r)) ** (1/r)
303
304 def fisch_expected(l,r):
305     return r * 2**l
306
307 nr_trials_per_r = 100
308 results_expected_number = [expected_number_of_queries(l,r) for r in
    rs]
309 fisch_expected_number = [fisch_expected(l,r) for r in rs]
310 results_fischlin = []
311 results_nc = []
312
313 for r in rs:
314     print(r)
315     fischlin_results = []
316     nc_results = []
317     for i in range(nr_trials_per_r):
318         fischlin = Fischlin(oracle_length=l,repetitions_required=r)
319         fischlin_results.append(fischlin.run()[4])
320         nc = NewConstruction(oracle_length=l,repetitions_required=r)
321         nc_results.append(nc.run()[0])
322     results_fischlin.append(np.mean(fischlin_results))
323     results_nc.append(np.mean(nc_results))
324
325 #write results to file
326 with open("results.txt","w+") as f:
327     f.write("l,r,expected_fischlin,expected,fischlin,nc\n")
328     for i in range(len(rs)):
329         f.write(f"{l},{rs[i]},{fisch_expected_number[i]},{
    results_expected_number[i]},{results_fischlin[i]},{results_nc[i
    ]}\n")
330
331 # Read results from file, and plot
332 fname = "results.txt"
333 with open(fname,"r") as f:
334     lines = f.readlines()
335     lines = lines[1:]
336     results = [list(map(float,line.strip().split(","))) for line in
    lines]
337     l = int(results[0][0])
338     rs = [int(result[1]) for result in results]
339     expected_fischlin = [result[2] for result in results]
340     expected = [result[3] for result in results]
```

```
341     fischlin = [result[4] for result in results]
342     nc = [result[5] for result in results]
343
344 plt.plot(rs,fischlin,label="Fischlin")
345 plt.plot(rs,nc,label="New Construction")
346 plt.plot(rs,expected,label="Expected for New Construction",linestyle
    ="--",color="orange")
347 plt.plot(rs,expected_fischlin,label="Expected for Fischlin",
    linestyle="--",color="blue")
348 plt.xlabel("r",fontsize=12)
349 plt.ylabel("Mean number of queries",fontsize=12)
350 plt.title(f"Mean number of queries for l={l} as a function of r",
    fontsize=14)
351 plt.legend()
352 plt.show()
```

Listing A.1: Python code for implementing and comparing protocols mentioned in this report.